Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO




# LABORATORIO N° 12

# Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

## DESARROLLO DE APLICACIONES EMPRESARIALES:

### Capacidades

1. **Identificar el papel de AJAX y de los servicios web RESTful (métodos HTTP, JSON y códigos de estado) en la comunicación entre React y Django.**
2. **Consumir la API REST de la Semana 8 desde React con Axios y useEffect, manejando los estados de carga y de error, con CORS configurado en Django.**
3. **Crear, actualizar, eliminar y buscar registros con POST, PUT o PATCH, DELETE y parámetros desde el panel administrativo del proyecto integrador.**

### I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

### II. Fundamento teórico

- Revise el material de la Semana 12 (Peticiones Ajax con ReactJS: AJAX y servicios web RESTful, JSON, métodos HTTP y códigos de estado, Fetch y Axios, GET con useEffect, estados de carga y error, POST, PUT y DELETE, y CORS), además de la Semana 8 (Django REST Framework) y la Semana 11 (comunicación entre componentes y useEffect) antes del desarrollo del laboratorio.

### III. Normas empleadas

- No aplica

### IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, Django REST Framework y django-cors-headers (pip install django-cors-headers), Node.js (versión LTS, que incluye npm), un editor de código (Visual Studio Code), un navegador web con sus herramientas de desarrollo (pestaña Network), un cliente de pruebas de API (Thunder Client, Postman o curl) y una cuenta de GitHub para el control de versiones.
- La API REST desarrollada en la Semana 8, el proyecto de React (carpeta frontend) de las Semanas 9 a 11 y el panel administrativo del proyecto integrador que se construye con Django y React (un panel propio, no el Django Admin).

### V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

### VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (api.js y los componentes .jsx), de la pestaña Network del navegador mostrando el método HTTP y el código de estado, de la página en el navegador y de la terminal cuando se ejecuten comandos.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

### VII. Procedimiento

Hasta la Semana 11, el front-end de React trabajó con datos escritos a mano. En esta sesión React se conecta con Django: pide los datos a la API REST de la Semana 8 con una petición Ajax, muestra la respuesta en la pantalla sin recargar la página y envía los cambios al servidor con POST, PUT y DELETE. Django y React son dos aplicaciones distintas que se ejecutan a la vez (puertos 8000 y 5173) y que se comunican por HTTP.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante resuelve un ejercicio simple sobre un recurso simple de su API de la Semana 8 (el mismo con el que probó el CRUD en esa sesión): lo lista, muestra los estados de carga y error, y crea, modifica y elimina registros, dentro de la carpeta src/practica. En la segunda parte, aplica lo mismo en el panel administrativo de su proyecto integrador, con su entidad principal, cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que React no reemplaza a Django: Django entrega y valida los datos (serializers y ViewSets) y React los pide y los dibuja. Lo que antes se resolvía con un Template y la recarga de la página, ahora se resuelve con una petición Ajax y un cambio de estado. En clase la teoría se demuestra con la API de productos de una cafetería; cada estudiante la traslada a su panel administrativo con ayuda de la siguiente tabla de equivalencias.

| **Concepto trabajado en clase** | **Equivalente en su panel administrativo** |
|---|---|
| Listar productos con GET /api/productos/ (en clase: la API de productos de la cafetería) | El GET a la lista de su entidad principal, por ejemplo /api/<recurso>/, con la API que creó en la Semana 8. |
| Estados de carga y de error (en clase: Cargando... y el error con Django apagado) | Los mismos tres estados en el listado de su panel, probados apagando el servidor de Django. |
| POST con el error 400 de validate_precio (en clase: crear un producto) | El POST de su formulario de alta, probando la validación validate_<campo> de su serializer de la Semana 8. |
| DELETE que responde 204 (en clase: eliminar un producto y quitarlo del estado) | El DELETE de un registro de su entidad principal, y la actualización de la lista en pantalla. |

**Usuario (carga la pantalla o hace clic)**

**→ Componente de React (useEffect o evento)**

**→ Axios (api.get, api.post, api.patch o api.delete)**

**→ Django: URL → Router → ViewSet (con CORS habilitado)**

**→ Serializer (valida y convierte a JSON) → Django ORM → SQLite**

**→ Response (JSON y código de estado)**

**→ setState (registros, cargando y error)**

**→ React vuelve a dibujar la pantalla**

Al finalizar el laboratorio, el estudiante deberá ser capaz de configurar CORS en Django, consumir una API REST desde React con Axios y useEffect, mostrar los estados de carga y de error, crear, modificar y eliminar registros con POST, PUT o PATCH y DELETE, enviar parámetros de

búsqueda, y de aplicar todo ello en el panel administrativo de su proyecto integrador y publicarlo en GitHub.

### Enunciados

## PARTE 1 — Ajax con un recurso simple de la API de la Semana 8

## Ejercicio 1 — Verificar la API y reconocer la petición

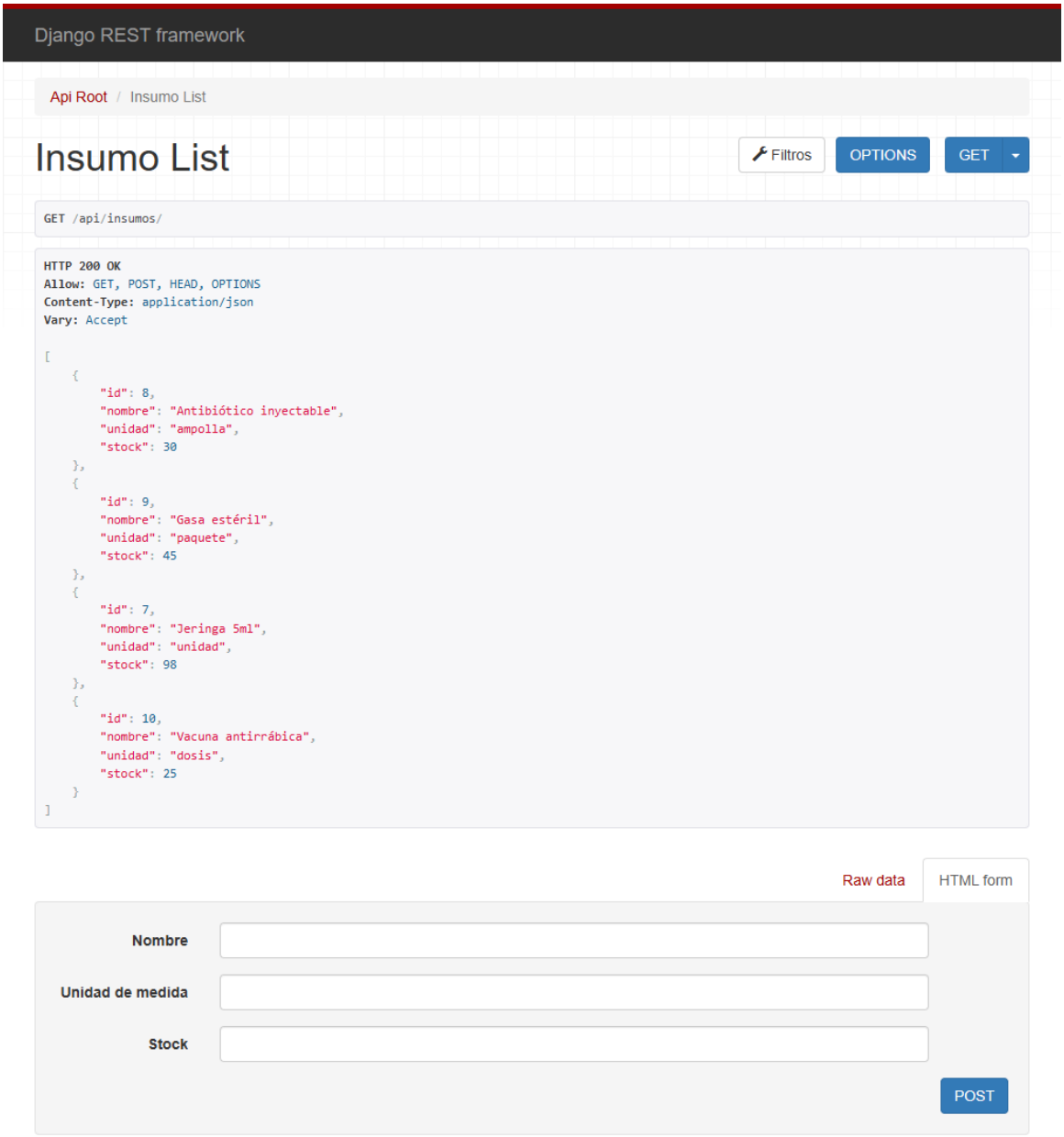
Levante Django con python manage.py runserver y abra en el navegador la URL de lista de un recurso simple de su API de la Semana 8. Documente la URL, el método HTTP, el código de estado y un fragmento del JSON. Complete una tabla con las operaciones CRUD de ese recurso, con estas columnas: operación, método HTTP, URL y código de estado esperado (200, 201, 204, 400 o 404). Explique con sus palabras qué diferencia hay entre esta respuesta y la de un Template de Django.

**Respuesta:**

Levanté Django con el servidor de desarrollo y abrí en el navegador la lista del recurso insumos: método GET, dirección http://127.0.0.1:8000/api/insumos/, código 200. La respuesta es una lista en JSON con el id, el nombre, la unidad y el stock de cada insumo. Tabla de operaciones del recurso:

| Operación | Método HTTP | URL | Código esperado |
|---|---|---|---|
| Listar | GET | /api/insumos/ | 200 |
| Ver uno | GET | /api/insumos/<id>/ | 200 (404 si no existe) |
| Crear | POST | /api/insumos/ | 201 (400 si no cumple la regla) |
| Modificar | PUT o PATCH | /api/insumos/<id>/ | 200 (400 o 404) |
| Eliminar | DELETE | /api/insumos/<id>/ | 204 (404 si no existe) |

La diferencia con un Template de Django es que la API no entrega una página lista para mostrar, sino solo los datos en JSON junto con un código de estado. Quien decide cómo dibujarlos es React; el Template, en cambio, mezcla los datos con el HTML en el servidor y devuelve la página completa.


Django REST framework
Api Root / Insumo List
Insumo List
Filtros
OPTIONS
GET
GET /api/insumos/
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
[
{
"id": 8,
"nombre": "Antibiótico inyectable",
"unidad": "ampolla",
"stock": 30
},
{
"id": 9,
"nombre": "Gasa estéril",
"unidad": "paquete",
"stock": 45
},
{
"id": 7,
"nombre": "Jeringa 5ml",
"unidad": "unidad",
"stock": 98
},
{
"id": 10,
"nombre": "Vacuna antirrábica",
"unidad": "dosis",
"stock": 25
}
]
Raw data
HTML form
Nombre
Unidad de medida
Stock
POST

```
GET http://127.0.0.1:8000/api/insumos/   →   200 OK
[
  {
    "id": 8,
    "nombre": "Antibiótico inyectable",
    "unidad": "ampolla",
    "stock": 30
  },
  {
    "id": 9,
    "nombre": "Gasa estéril",
    "unidad": "paquete",
    "stock": 45
  },
  {
    "id": 7,
    "nombre": "Jeringa 5ml",
    "unidad": "unidad",
    "stock": 98
  },
  {
    "id": 10,
    "nombre": "Vacuna antirrábica",
    "unidad": "dosis",
    "stock": 25
  }
]
```

## Ejercicio 2 — Configurar CORS en Django

Con React ejecutándose en http://localhost:5173/, abra la consola del navegador y ejecute fetch con la URL de su recurso; documente el mensaje de error de CORS. Instale django-cors-headers (pip install django-cors-headers) y actualice requirements.txt; agregue corsheaders a INSTALLED_APPS, agregue corsheaders.middleware.CorsMiddleware en la parte alta de MIDDLEWARE y defina CORS_ALLOWED_ORIGINS con http://localhost:5173. Repita el fetch y documente la respuesta. Explique con sus palabras por qué ocurre el bloqueo.

**Respuesta:**

Con React abierto en http://localhost:5173/ ejecuté un fetch hacia la API desde la consola del navegador. El navegador lo bloqueó con un mensaje de CORS: no hay un encabezado que permita el acceso desde ese origen. Instalé django-cors-headers, lo agregué a las aplicaciones instaladas, puse su middleware en la parte alta y definí como origen permitido http://localhost:5173; también actualicé requirements.txt. Al repetir el fetch la respuesta fue 200 con la lista de insumos. El bloqueo ocurre porque el navegador considera otro origen a una dirección con distinto puerto, y por seguridad no deja que una página lea datos de otro origen si el servidor no lo autoriza expresamente. La petición sí llega a Django; lo que el navegador impide es que la página lea la respuesta.

```
Consola del navegador en http://localhost:5173/ (antes de configurar CORS)
> fetch('http://127.0.0.1:8000/api/insumos/')
Access to fetch at 'http://127.0.0.1:8000/api/insumos/' from origin 'http://localhost:5173' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
Failed to load resource: net::ERR_FAILED
Resultado del fetch: ERROR: Failed to fetch
```

```
# config/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'vet',  # Reserva de citas de la veterinaria (Laboratorio 02)
    'biblioteca', # Catalogo de biblioteca comunitaria (Laboratorio 03 - Parte 2)
    'rest_framework',  # Django REST Framework (Laboratorio 08)
    'corsheaders',  # Permite que React (otro puerto) llame a la API
    'chamba',  # Panel administrativo del proyecto integrador Chamba PE
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',  # va en la parte alta
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

```
// config/settings.py (final del archivo)
# Origen de React (Vite) autorizado para llamar a la API
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5173',
]


// requirements.txt
asgiref==3.12.1
django-cors-headers==4.9.0
Django==5.2.17
djangorestframework==3.18.3
sqlparse==0.6.0
tzdata==2026.3
```

```
Consola del navegador en http://localhost:5173/ (con CORS configurado)
> fetch('http://127.0.0.1:8000/api/insumos/')
Resultado: OK 200 — 4 insumos recibidos
```

## Ejercicio 3 — Instalar Axios y crear una instancia

En la carpeta frontend ejecute npm install axios y cree src/api.js con axios.create, definiendo baseURL con la dirección de su API (por ejemplo, http://127.0.0.1:8000/api) y un timeout. Documente el código y el archivo package.json. Explique qué ventaja tiene centralizar la dirección del servidor en un solo archivo.

**Respuesta:**

En la carpeta frontend instalé Axios con npm y creé el archivo api.js con una instancia que tiene la dirección base de la API y un tiempo máximo de espera de 5 segundos. El package.json muestra axios entre las dependencias. Centralizar la dirección del servidor en un solo archivo tiene la ventaja de que, si la dirección cambia (por ejemplo, al pasar a producción), se modifica en un único lugar y no en cada componente; además evita errores de escritura repetidos.

```
// api.js
import axios from 'axios'

// Instancia única: la dirección del servidor se escribe solo aquí
const api = axios.create({
  baseURL: 'http://127.0.0.1:8000/api',
  timeout: 5000,
})

// Convierte el error de Axios en un texto para mostrar al usuario
export function textoError(err) {
  const datos = err.response && err.response.data
  if (datos && typeof datos === 'object') {
    return Object.entries(datos)
      .map(([campo, mensajes]) => `${campo}: ${[].concat(mensajes).join(' ')}`)
      .join(' | ')
  }
  return 'No se pudo conectar con el servidor.'
}

export default api
```

```
// frontend/package.json (dependencies)
{
  "dependencies": {
    "axios": "^1.21.1",
    "react": "^19.3.1",
    "react-dom": "^19.3.1"
  }
}
```

## Ejercicio 4 — Solicitud GET con useEffect

En src/practica, cree el componente ListaPractica.jsx que pida la lista de su recurso con await api.get(...) dentro de un useEffect con arreglo de dependencias vacío, guarde la respuesta en el estado y la muestre con map() y key basada en el id. Documente el código, la página y la pestaña Network con el método GET y el código 200. Explique por qué la petición va dentro de useEffect y no en el cuerpo del componente, y por qué la función del efecto no se declara async.

**Respuesta:**

El componente ListaPractica pide la lista de insumos con la instancia api dentro de un efecto con arreglo de dependencias, guarda la respuesta en el estado y la dibuja con map y una key basada en el id. En la pestaña Network se ve la petición GET con el código 200. La petición va dentro del efecto porque el cuerpo del componente se ejecuta en cada dibujo: si la petición estuviera ahí, cada respuesta cambiaría el estado, se dibujaría otra vez y volvería a pedir los datos sin parar. El efecto la hace solo cuando corresponde. La función del efecto no se declara async porque un efecto solo puede devolver nada o una función de limpieza, y una función async devuelve una promesa; por eso dentro del efecto se declara una función async aparte y se la llama.

```jsx
// practica/ListaPractica.jsx
import { useEffect, useState } from 'react'
import api, { textoError } from '../api.js'

const formularioVacio = { nombre: '', unidad: 'unidad', stock: '' }

// Insumos leídos de la API de Django: lista, alta, cambio de stock y baja
function ListaPractica() {
  const [insumos, setInsumos] = useState([])
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState(false)
  const [buscar, setBuscar] = useState('')
  const [form, setForm] = useState(formularioVacio)
  const [errorForm, setErrorForm] = useState('')

  // La petición va en un efecto: se ejecuta después de dibujar y cada vez que cambia buscar
  useEffect(() => {
    async function cargar() {
      try {
        const respuesta = await api.get('/insumos/', { params: { search: buscar } })
        setInsumos(respuesta.data)
        setError(false)
      } catch (err) {
        console.log('Error al cargar los insumos:', err.message)
        setError(true)
      } finally {
        setCargando(false)
      }
    }
    cargar()
  }, [buscar])

  function manejarCambio(e) {
    setForm({ ...form, [e.target.name]: e.target.value })
  }

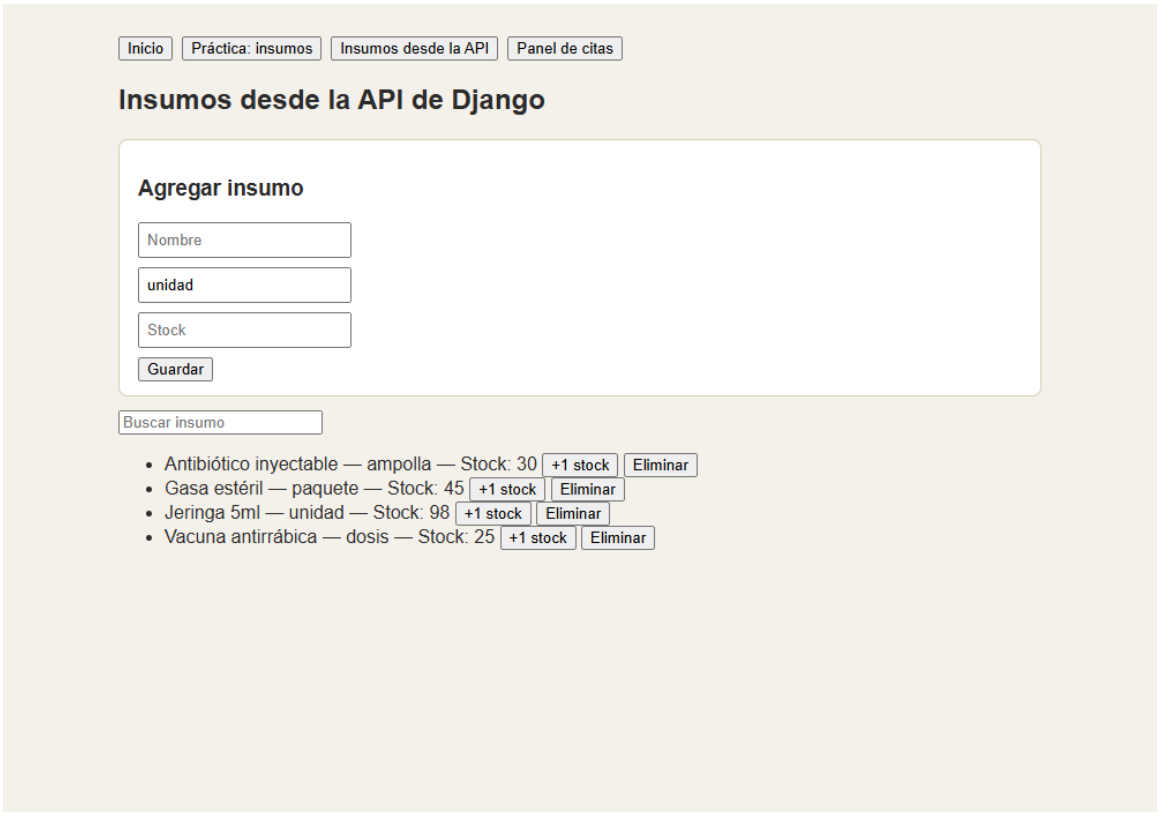
  async function crear(e) {
    e.preventDefault()
    try {
      const respuesta = await api.post('/insumos/', {
        nombre: form.nombre,
        unidad: form.unidad,
        stock: Number(form.stock),
      })
      setInsumos([...insumos, respuesta.data])
      setForm(formularioVacio)
      setErrorForm('')
    } catch (err) {
      setErrorForm(textoError(err))
    }
  }

  async function eliminar(id) {
    try {
      await api.delete(`/insumos/${id}/`)
      setInsumos(insumos.filter((i) => i.id !== id))
    } catch (err) {
      setErrorForm(textoError(err))
    }
  }

  async function sumarStock(insumo) {
    try {
      const respuesta = await api.patch(`/insumos/${insumo.id}/`, { stock: insumo.stock + 1 })
      setInsumos(insumos.map((i) => (i.id === insumo.id ? respuesta.data : i)))
    } catch (err) {
      setErrorForm(textoError(err))
    }
  }

  return (
    <div className="seccion-vet">
      <h2>Insumos desde la API de Django</h2>

      <form onSubmit={crear} className="formulario">
        <h3>Agregar insumo</h3>
        <input name="nombre" placeholder="Nombre" value={form.nombre} onChange={manejarCambio} />
        <input name="unidad" placeholder="Unidad" value={form.unidad} onChange={manejarCambio} />
        <input name="stock" type="number" placeholder="Stock" value={form.stock} onChange={manejarCambio} />
        <button type="submit">Guardar</button>
```





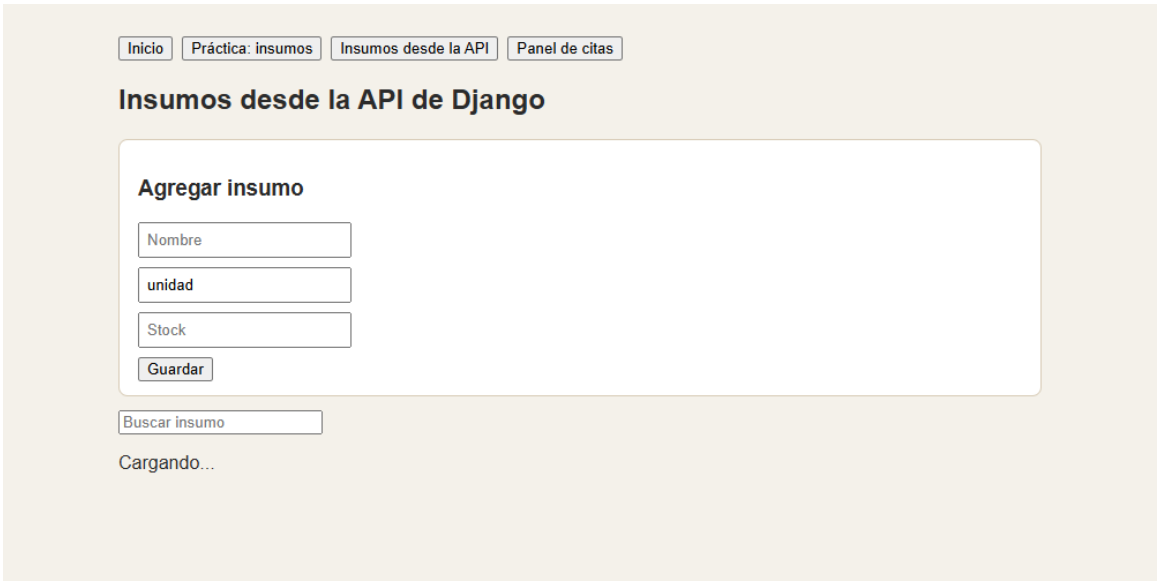


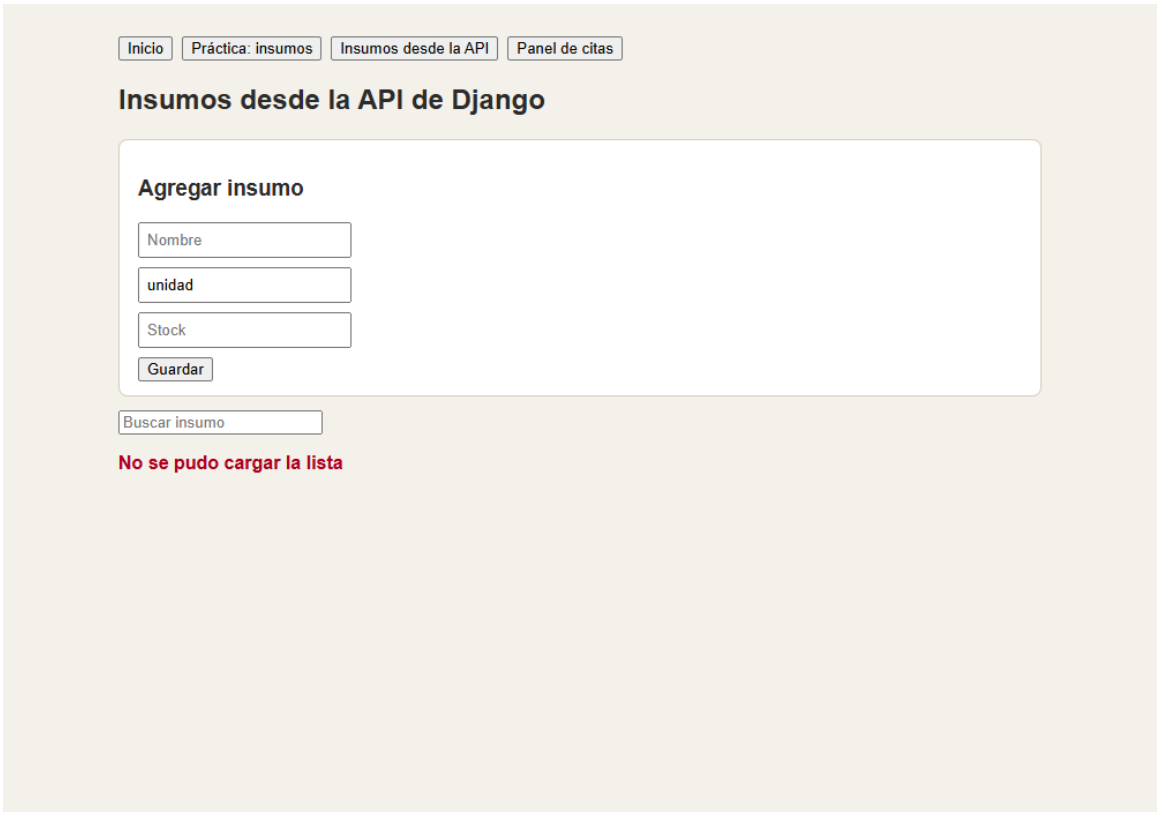
## Ejercicio 5 — Estados de carga y de error

Agregue los estados cargando y error, con try, catch y finally, y muestre los mensajes Cargando... y No se pudo cargar la lista. Muestre también un mensaje cuando la lista esté vacía. Pruebe el error apagando Django y documente la pantalla y el mensaje de la consola. Explique qué muestra la pantalla en cada uno de los tres estados y por qué es importante no dejarla en blanco.

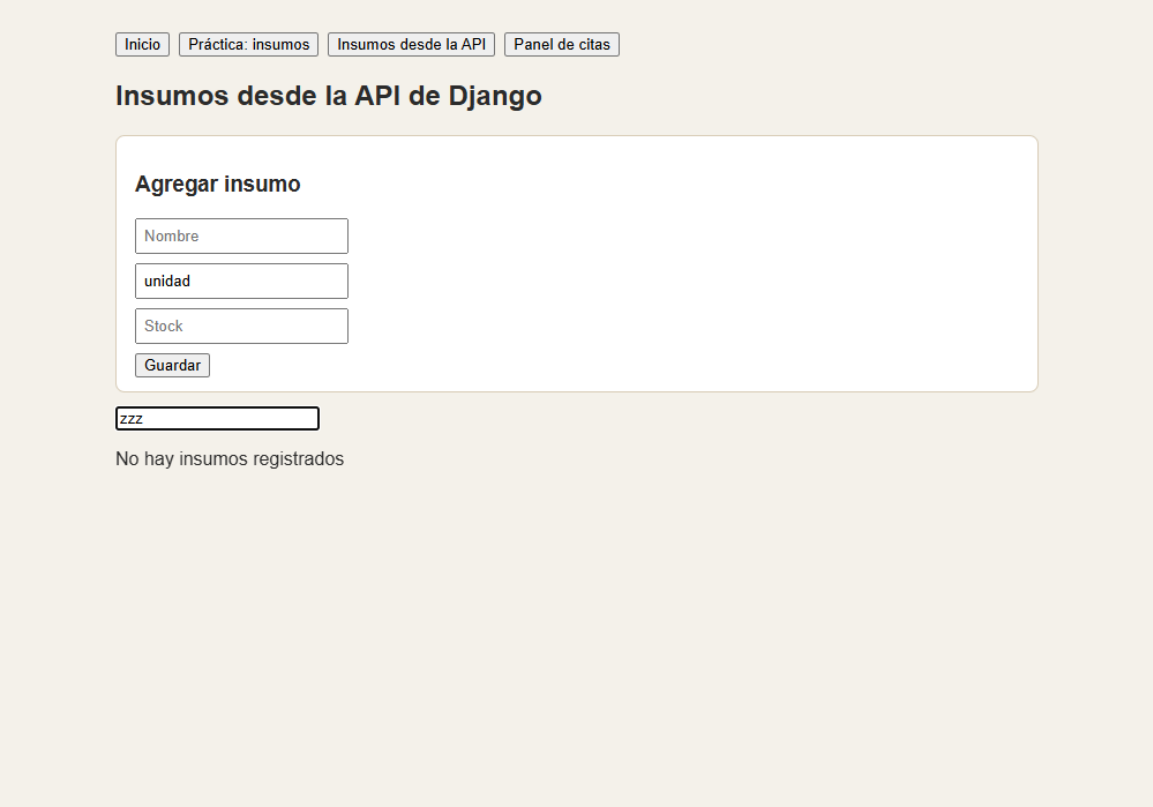
**Respuesta:**

Agregué los estados cargando y error con try, catch y finally. Mientras llega la respuesta la pantalla muestra Cargando...; si la petición falla muestra No se pudo cargar la lista; si responde bien pero no hay datos muestra No hay insumos registrados. Para probar el error apagué Django: la pantalla mostró el mensaje de error y la consola registró el fallo de red. Es importante no dejar la pantalla en blanco porque la persona no sabría si la aplicación está cargando, si falló o si simplemente no hay datos.




```
Consola del navegador con Django apagado
Error al cargar los insumos: Network Error
Failed to load resource: net::ERR_CONNECTION_REFUSED
Error al cargar los insumos: Network Error
Failed to load resource: net::ERR_CONNECTION_REFUSED
```

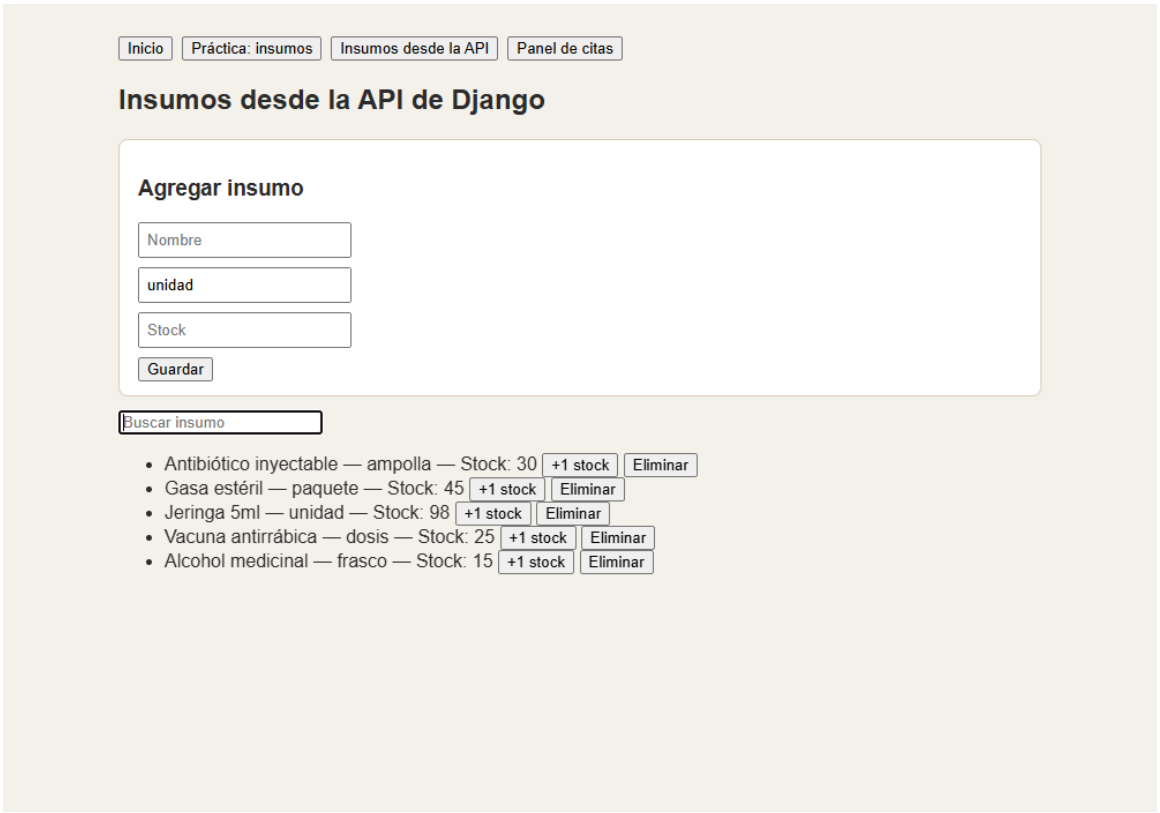



## Ejercicio 6 — Solicitud POST con un formulario controlado

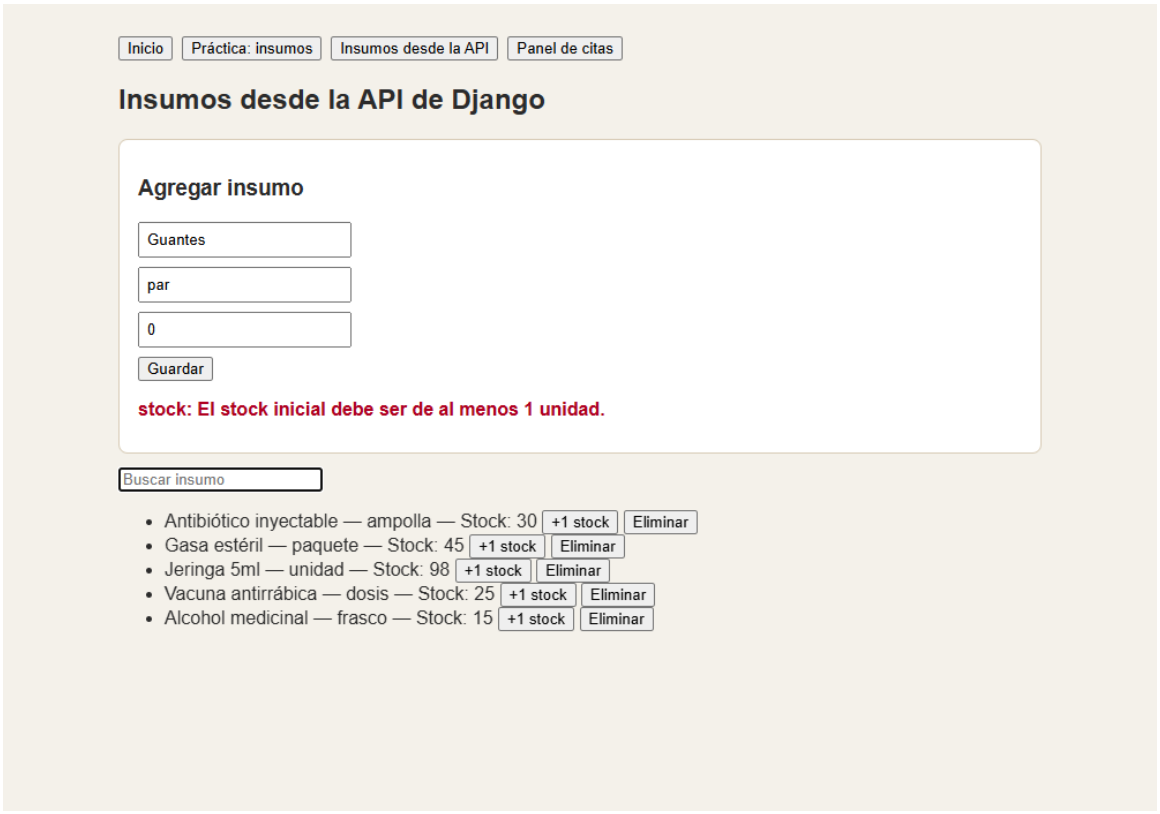
Agregue un formulario controlado que cree un registro con api.post(...) y agregue respuesta.data al estado con un arreglo nuevo. Documente la pestaña Network con el método POST y el código 201. Envíe un dato que incumpla la regla de validación de su serializer (Semana 8, ejercicio 3), documente la respuesta 400 y muestre en pantalla el mensaje recibido en err.response.data.

**Respuesta:**

Agregué un formulario controlado para crear insumos con la instancia api y un POST; la respuesta se agrega al estado con un arreglo nuevo. Con un insumo válido la pestaña Network muestra POST con el código 201 y el insumo aparece sin recargar la página. Con un stock de cero, que incumple la regla de mi serializer de la Semana 8, la API responde 400 y la pantalla muestra el mensaje recibido: el stock inicial debe ser de al menos 1 unidad.


Inicio
Práctica: insumos
Insumos desde la API
Panel de citas
Insumos desde la API de Django
Agregar insumo
Nombre
unidad
Stock
Guardar
Buscar insumo
Antibiótico inyectable — ampolla — Stock: 30 +1 stock Eliminar
Gasa estéril — paquete — Stock: 45 +1 stock Eliminar
Jeringa 5ml — unidad — Stock: 98 +1 stock Eliminar
Vacuna antirrábica — dosis — Stock: 25 +1 stock Eliminar
Alcohol medicinal — frasco — Stock: 15 +1 stock Eliminar


Pestaña Network: POST válido
Método Est. URL
OPTIONS 200 http://127.0.0.1:8000/api/insumos/
POST 201 http://127.0.0.1:8000/api/insumos/


Inicio
Práctica: insumos
Insumos desde la API
Panel de citas
Insumos desde la API de Django
Agregar insumo
Guantes
par
0
Guardar
stock: El stock inicial debe ser de al menos 1 unidad.
Buscar insumo
Antibiótico inyectable — ampolla — Stock: 30 +1 stock Eliminar
Gasa estéril — paquete — Stock: 45 +1 stock Eliminar
Jeringa 5ml — unidad — Stock: 98 +1 stock Eliminar
Vacuna antirrábica — dosis — Stock: 25 +1 stock Eliminar
Alcohol medicinal — frasco — Stock: 15 +1 stock Eliminar

```
Pestaña Network: POST que incumple la regla del serializer
Método  Est. URL
POST    400  http://127.0.0.1:8000/api/insumos/
```

## Ejercicio 7 — Solicitudes DELETE y PATCH o PUT

Agregue a cada fila un botón Eliminar que llame a api.delete(...) y quite el registro del estado con filter() al recibir la respuesta 204. Agregue un botón o un campo que modifique un dato con api.patch(...) (o api.put(...)) y actualice el estado con map(). Documente en Network los métodos y códigos obtenidos (204 y 200) y la página antes y después de cada acción.

**Respuesta:**

Cada fila tiene un botón Eliminar que llama a DELETE y, al recibir el código 204, quita el insumo del estado con filter. También agregué un botón de más uno de stock que llama a PATCH y reemplaza el insumo en el estado con map. En Network se ven el PATCH con 200 y el DELETE con 204; las capturas de la página muestran el estado antes y después de cada acción.

```jsx
// practica/ListaPractica.jsx
import { useEffect, useState } from 'react'
import api, { textoError } from '../api.js'

const formularioVacio = { nombre: '', unidad: 'unidad', stock: '' }

// Insumos leídos de la API de Django: lista, alta, cambio de stock y baja
function ListaPractica() {
  const [insumos, setInsumos] = useState([])
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState(false)
  const [buscar, setBuscar] = useState('')
  const [form, setForm] = useState(formularioVacio)
  const [errorForm, setErrorForm] = useState('')

  // La petición va en un efecto: se ejecuta después de dibujar y cada vez que cambia buscar
  useEffect(() => {
    async function cargar() {
      try {
        const respuesta = await api.get('/insumos/', { params: { search: buscar } })
        setInsumos(respuesta.data)
        setError(false)
      } catch (err) {
        console.log('Error al cargar los insumos:', err.message)
        setError(true)
      } finally {
        setCargando(false)
      }
    }
    cargar()
  }, [buscar])

  function manejarCambio(e) {
    setForm({ ...form, [e.target.name]: e.target.value })
  }

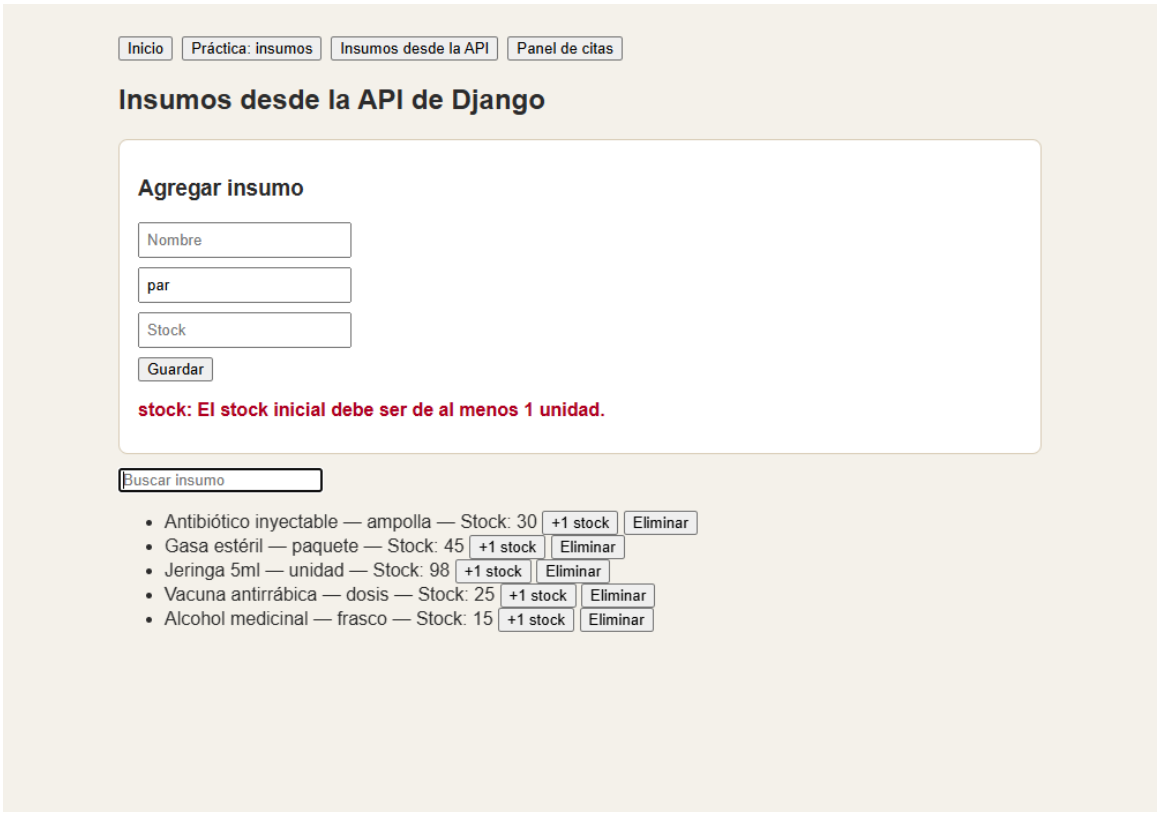
  async function crear(e) {
    e.preventDefault()
    try {
      const respuesta = await api.post('/insumos/', {
        nombre: form.nombre,
        unidad: form.unidad,
        stock: Number(form.stock),
      })
      setInsumos([...insumos, respuesta.data])
      setForm(formularioVacio)
      setErrorForm('')
    } catch (err) {
      setErrorForm(textoError(err))
    }
  }

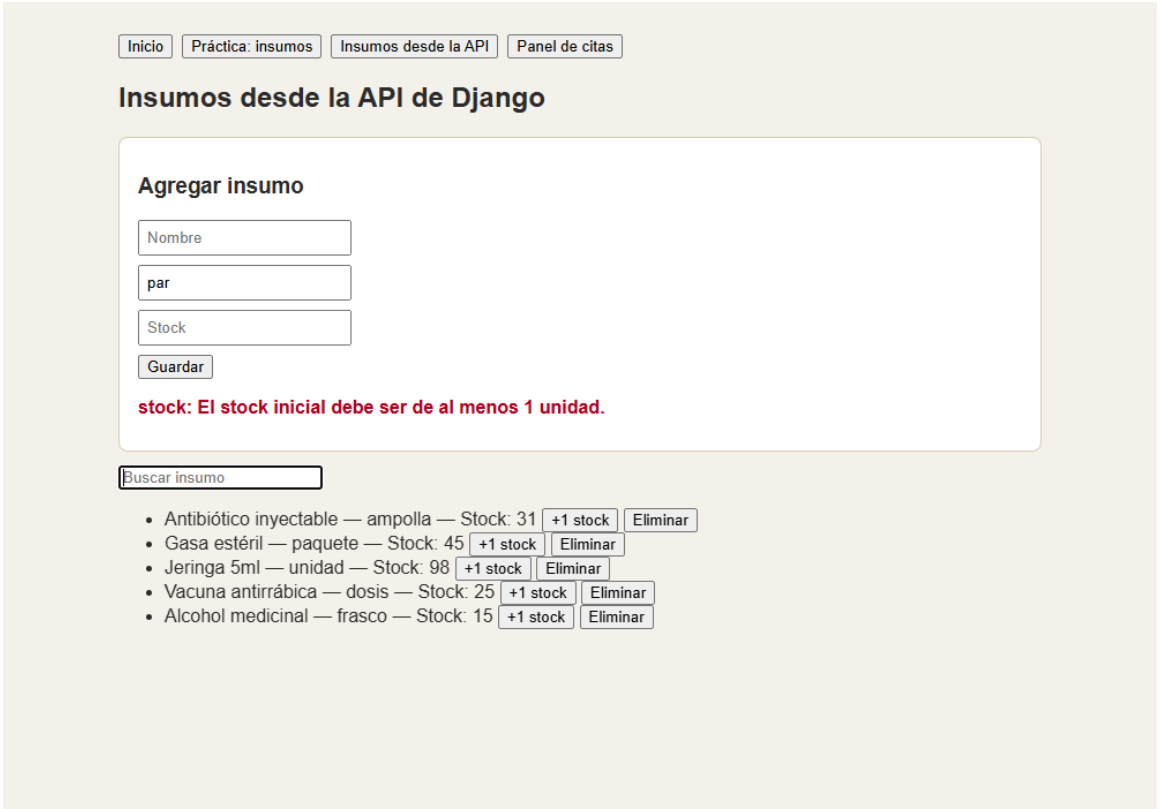
  async function eliminar(id) {
    try {
      await api.delete(`/insumos/${id}/`)
      setInsumos(insumos.filter((i) => i.id !== id))
    } catch (err) {
      setErrorForm(textoError(err))
    }
  }

  async function sumarStock(insumo) {
    try {
      const respuesta = await api.patch(`/insumos/${insumo.id}/`, { stock: insumo.stock + 1 })
      setInsumos(insumos.map((i) => (i.id === insumo.id ? respuesta.data : i)))
    } catch (err) {
      setErrorForm(textoError(err))
    }
  }

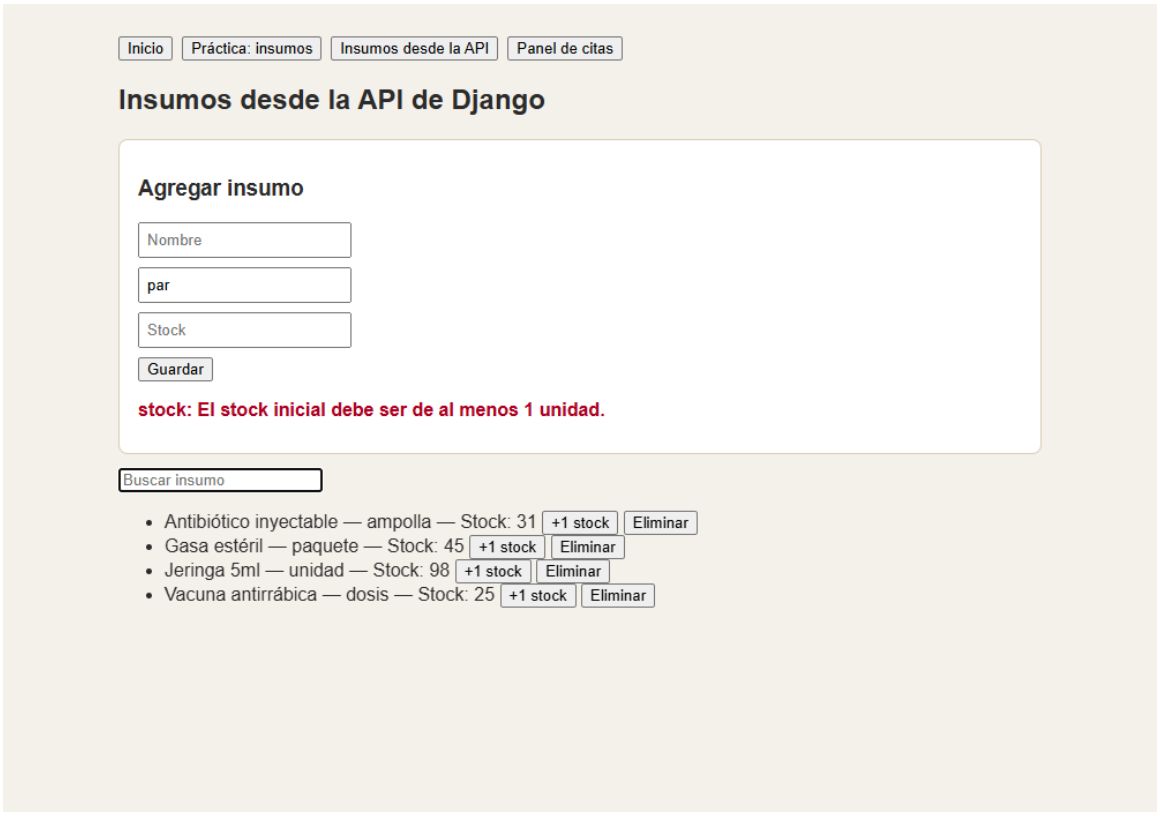
  return (
    <div className="seccion-vet">
      <h2>Insumos desde la API de Django</h2>

      <form onSubmit={crear} className="formulario">
        <h3>Agregar insumo</h3>
        <input name="nombre" placeholder="Nombre" value={form.nombre} onChange={manejarCambio} />
        <input name="unidad" placeholder="Unidad" value={form.unidad} onChange={manejarCambio} />
        <input name="stock" type="number" placeholder="Stock" value={form.stock} onChange={manejarCambio} />
        <button type="submit">Guardar</button>
```




```
Pestaña Network: PATCH del stock
Método  Est. URL
OPTIONS 200  http://127.0.0.1:8000/api/insumos/8/
PATCH   200  http://127.0.0.1:8000/api/insumos/8/
```



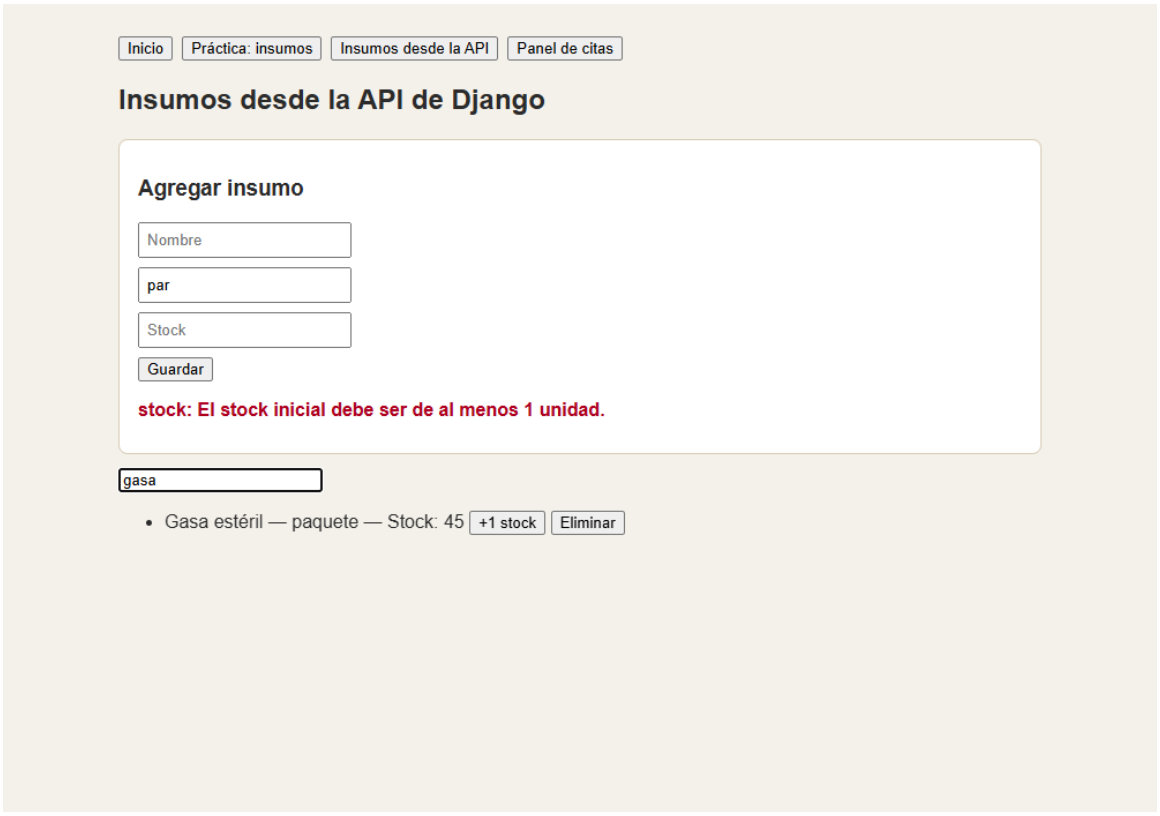

```
Pestaña Network: DELETE de un insumo
Método  Est. URL
OPTIONS 200  http://127.0.0.1:8000/api/insumos/11/
DELETE  204  http://127.0.0.1:8000/api/insumos/11/
```

## Ejercicio 8 — Parámetros de búsqueda y verificación

Agregue un campo de búsqueda controlado que envíe el texto como parámetro (params: { search: buscar }) y recargue la lista con un useEffect cuyo arreglo de dependencias sea [buscar]. Documente la URL generada con ?search= en la pestaña Network. Ejecute npm run lint y npm run build y documente la salida. Explique con sus palabras qué hace cada pieza (React, Axios, el ViewSet, el serializer y el ORM) en el recorrido de una petición, y por qué su API todavía está abierta a cualquier persona.

**Respuesta:**

El campo de búsqueda controlado envía el texto como parámetro y la lista se vuelve a pedir con un efecto cuya dependencia es el texto buscado. En Network la dirección generada es http://127.0.0.1:8000/api/insumos/?search=gasa. npm run lint no reportó errores y npm run build terminó correctamente. En el recorrido de una petición, React muestra la pantalla y decide cuándo pedir los datos; Axios envía la petición HTTP y recibe la respuesta; el ViewSet recibe la petición y decide qué operación realizar; el serializer valida los datos que llegan y los convierte a JSON; y el ORM habla con la base de datos SQLite. Mi API todavía está abierta a cualquier persona porque no tiene autenticación ni permisos: cualquiera que conozca la dirección puede leer, crear, modificar o eliminar registros.




```
Pestaña Network: búsqueda con parámetro
Método  Est. URL
GET     200  http://127.0.0.1:8000/api/insumos/?search=g
GET     200  http://127.0.0.1:8000/api/insumos/?search=ga
GET     200  http://127.0.0.1:8000/api/insumos/?search=gas
GET     200  http://127.0.0.1:8000/api/insumos/?search=gasa
```

```
PS ...> npm run lint

> frontend@0.0.0 lint
> eslint .
```

```
PS ...> npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.5 building client environment for production...
transforming...
✓ 104 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.47 kB │ gzip:  0.30 kB
dist/assets/index-xrUpEBpY.css   1.27 kB │ gzip:  0.47 kB
dist/assets/index-DY1ZNk4g.js  297.92 kB │ gzip: 95.59 kB

✓ built in 182ms
```

## PARTE 2 — Ajax y servicios web en el panel administrativo del proyecto integrador

A partir de aquí trabajará en el panel administrativo de su proyecto integrador. El objetivo es reemplazar los datos escritos a mano de la Semana 11 por datos reales de su API: el listado se carga con GET, el formulario de alta envía POST, las filas se editan con PUT o PATCH y se eliminan con DELETE, y el buscador envía parámetros a la API, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

## Ejercicio 9 — Identificar el recurso del panel y sus operaciones

Complete la tabla de equivalencias con su proyecto: indique la entidad principal de su panel, la URL de su API, sus campos, la regla de validación (validate_<campo>) que usará para probar el error 400, los filtros que ya tiene la API (?search=, ?ordering=, ?campo=) y las operaciones que su panel necesita (listar, crear, editar y eliminar). Si algún filtro o regla no existe en su API, agréguelo antes de continuar.

**Respuesta:**

Mi proyecto integrador es Chamba PE y la entidad principal de su panel administrativo es el prestador.
Tabla de equivalencias:

| **Elemento** | **En el panel de Chamba PE** |
|---|---|
| Entidad principal | Prestador |
| URL de la API | http://127.0.0.1:8000/api/chamba/prestadores/ |
| Campos | id, nombres, apellidos, rubro, tarifa (decimal, llega como texto) y estado |
| Regla de validación | validate_tarifa: la tarifa debe ser mayor que cero (responde 400) |
| Filtros de la API | ?search= (nombres y apellidos), ?estado= y ?ordering= (tarifa o apellidos) |
| Operaciones del panel | Listar, crear, editar, aprobar o suspender (PATCH) y eliminar |

El panel de la Semana 11 trabajaba con datos escritos a mano y la API del proyecto todavía no tenía este recurso, así que antes de continuar lo agregué al proyecto Django del curso como la app chamba: el modelo Prestador, su serializer con la regla validate_tarifa, un ModelViewSet con búsqueda, filtro por estado y orden, y la ruta /api/chamba/prestadores/. Los seis prestadores de ejemplo se cargan con el script cargar_chamba.py. Django corre en el puerto 8000 y React en el 5173, con CORS configurado como en la Parte 1.

```
# chamba/models.py
# chamba/models.py
# Prestador de Chamba PE: la entidad principal del panel administrativo.

from django.db import models


class Prestador(models.Model):
    RUBROS = [
        ('Gasfitería', 'Gasfitería'),
        ('Electricidad', 'Electricidad'),
        ('Costura', 'Costura'),
        ('Carpintería', 'Carpintería'),
        ('Primeros auxilios', 'Primeros auxilios'),
    ]
    ESTADOS = [
        ('Pendiente', 'Pendiente'),
        ('Aprobado', 'Aprobado'),
        ('Suspendido', 'Suspendido'),
    ]

    nombres = models.CharField(max_length=80)
    apellidos = models.CharField(max_length=80)
    rubro = models.CharField(max_length=30, choices=RUBROS)
    tarifa = models.DecimalField('Tarifa por visita (S/)', max_digits=7, decimal_places=2)
    estado = models.CharField(max_length=20, choices=ESTADOS, default='Pendiente')

    class Meta:
        ordering = ['apellidos', 'nombres']

    def __str__(self):
        return f'{self.nombres} {self.apellidos} ({self.rubro})'

# chamba/serializers.py
# chamba/serializers.py
# Convierte los prestadores a JSON y valida lo que llega del panel.

from rest_framework import serializers

from .models import Prestador


class PrestadorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Prestador
        fields = ['id', 'nombres', 'apellidos', 'rubro', 'tarifa', 'estado']

    # Regla de negocio: la tarifa por visita debe ser mayor que cero.
    def validate_tarifa(self, value):
        if value <= 0:
            raise serializers.ValidationError('La tarifa debe ser mayor que cero.')
        return value

# chamba/api_views.py
# chamba/api_views.py
# CRUD de prestadores para el panel de React: GET, POST, PUT, PATCH y DELETE.

from rest_framework import filters, viewsets

from .models import Prestador
from .serializers import PrestadorSerializer


class PrestadorViewSet(viewsets.ModelViewSet):
    serializer_class = PrestadorSerializer
    # ?search=texto busca en nombres y apellidos; ?ordering=tarifa ordena.
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['nombres', 'apellidos']
    ordering_fields = ['tarifa', 'apellidos']

    def get_queryset(self):
        prestadores = Prestador.objects.all()
        # Filtro por campo: /api/chamba/prestadores/?estado=Aprobado
        estado = self.request.query_params.get('estado')
        if estado:
            prestadores = prestadores.filter(estado=estado)
        return prestadores

# chamba/api_urls.py
# chamba/api_urls.py
```

```
GET http://127.0.0.1:8000/api/chamba/prestadores/  →  200 OK
[
  {
    "id": 6,
    "nombres": "Rosa",
    "apellidos": "Díaz Huamán",
    "rubro": "Electricidad",
    "tarifa": "55.00",
    "estado": "Aprobado"
  },
  {
    "id": 3,
    "nombres": "Luis",
    "apellidos": "Gamarra Soto",
    "rubro": "Electricidad",
    "tarifa": "50.00",
    "estado": "Aprobado"
  },
  {
    "id": 5,
    "nombres": "Jorge",
    "apellidos": "Medina Salas",
    "rubro": "Carpintería",
    "tarifa": "70.00",
    "estado": "Suspendido"
  },
  {
    "id": 4,
    "nombres": "Ana",
    "apellidos": "Paredes Ríos",
    "rubro": "Primeros auxilios",
    "tarifa": "40.00",
    "estado": "Pendiente"
  },
  {
    "id": 1,
    "nombres": "Carlos",
    "apellidos": "Ramos Quispe",
    "rubro": "Gasfitería",
    "tarifa": "60.00",
    "estado": "Aprobado"
  },
  {
    "id": 2,
    "nombres": "María",
    "apellidos": "Torres Vega",
    "rubro": "Costura",
    "tarifa": "25.00",
    "estado": "Pendiente"
  }
]
```

```python
# config/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'vet',  # Reserva de citas de la veterinaria (Laboratorio 02)
    'biblioteca', # Catalogo de biblioteca comunitaria (Laboratorio 03 - Parte 2)
    'rest_framework',  # Django REST Framework (Laboratorio 08)
    'corsheaders',  # Permite que React (otro puerto) llame a la API
    'chamba',  # Panel administrativo del proyecto integrador Chamba PE
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',  # va en la parte alta
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

```
PS ...> Django y React a la vez
Terminal 1 (django_project/src): python manage.py runserver
  Starting development server at http://127.0.0.1:8000/

Terminal 2 (frontend): npm run dev

  VITE v8.3.5  ready in 253 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

## Checklist antes de continuar — Parte 2

| Criterio | Verificación |
|---|---|
| 1. django-cors-headers está instalado y configurado, requirements.txt está actualizado y Django (puerto 8000) y React (puerto 5173) se ejecutan a la vez. | Captura de settings.py, de requirements.txt y de las dos terminales. |
| 2. Axios está instalado y existe src/api.js con baseURL; las URL completas no se repiten en los componentes. | Código de api.js y package.json. |
| 3. El listado del panel se carga con GET dentro de un useEffect con arreglo de dependencias, desde la API y no desde datos escritos a mano. | Código + captura de Network con GET y 200. |
| 4. El panel muestra los estados de carga, de error (probado con Django apagado) y de lista vacía. | Capturas de los tres estados. |
| 5. El formulario de alta envía un POST; el registro nuevo aparece sin recargar la página y el estado se actualiza con un arreglo nuevo. | Código + captura de Network con POST y 201. |
| 6. Los errores de validación 400 de la API se muestran al usuario junto al formulario. | Captura del mensaje y de la respuesta 400. |
| 7. Existe edición con PUT o PATCH y eliminación con DELETE (204), con confirmación previa al eliminar. | Código + capturas de Network (200 y 204). |
| 8. El buscador o filtro del panel envía parámetros a la API (?search=, ?campo=) y recarga el listado cuando | URL generada + captura. |

| | |
|---|---|
| cambian. | |
| 9. La consola no muestra advertencias, npm run lint y npm run build terminan sin errores, y la terminal de Django no muestra errores 500. | Capturas de la consola y de las terminales. |

## Ejercicio 10 — Cargar el listado del panel desde la API

Reemplace los datos escritos a mano de la Semana 11 por una solicitud GET dentro de un useEffect, usando la instancia api de src/api.js. El componente padre guarda los registros, y también los estados cargando y error, y entrega los datos al listado por props. Si la API entrega decimales como texto, conviértalos con una función adaptadora. Documente el código, el listado en pantalla y la pestaña Network con el código 200.

**Respuesta:**

Reemplacé los datos escritos a mano de la Semana 11 por un GET dentro de un efecto con la instancia api de src/api.js. El componente padre PanelPrestadores guarda los prestadores y también los estados cargando y error, y entrega los datos al listado por props. Como la API entrega la tarifa como texto, una función adaptadora la convierte a número antes de guardarla. En Network se ve la petición GET con el código 200. Mientras llega la respuesta se muestra Cargando..., y con Django apagado aparece No se pudo cargar la lista de prestadores; si no hay resultados, el listado muestra No hay prestadores que coincidan.

```
# api.js
import axios from 'axios'

// Instancia única: la dirección del servidor se escribe solo aquí
const api = axios.create({
  baseURL: 'http://127.0.0.1:8000/api',
  timeout: 5000,
})

// Convierte el error de Axios en un texto para mostrar al usuario
export function textoError(err) {
  const datos = err.response && err.response.data
  if (datos && typeof datos === 'object') {
    return Object.entries(datos)
      .map(([campo, mensajes]) => `${campo}: ${[].concat(mensajes).join(' ')}`)
      .join(' | ')
  }
  return 'No se pudo conectar con el servidor.'
}

export default api

# components/PanelPrestadores.jsx
import { useEffect, useState } from 'react'
import api, { textoError } from '../api.js'
import PrestadorListado from './PrestadorListado.jsx'
import PrestadorForm from './PrestadorForm.jsx'
import ResumenPrestadores from './ResumenPrestadores.jsx'
import BuscadorPrestadores from './BuscadorPrestadores.jsx'
import FiltroEstadoPrestador from './FiltroEstadoPrestador.jsx'
import Aviso from './Aviso.jsx'

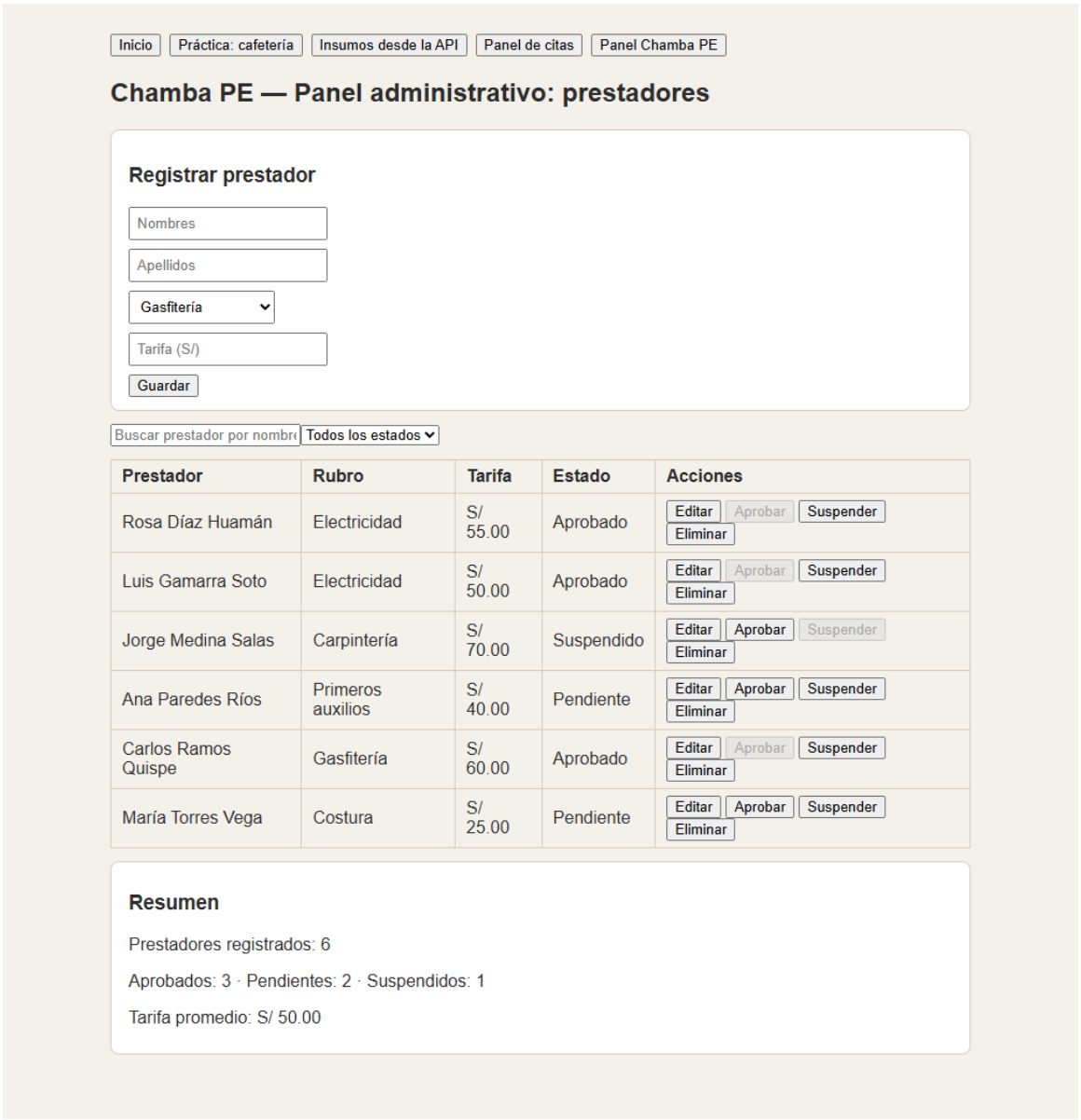
// Función adaptadora: la API entrega la tarifa como texto ("60.00") y el panel la usa como número
function adaptar(prestador) {
  return { ...prestador, tarifa: Number(prestador.tarifa) }
}

// Componente padre del panel de Chamba PE: guarda los prestadores y los estados de la petición
function PanelPrestadores() {
  const [prestadores, setPrestadores] = useState([])
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState(false)
  const [busqueda, setBusqueda] = useState('')
  const [estado, setEstado] = useState('')
  const [editando, setEditando] = useState(null)
  const [porEliminar, setPorEliminar] = useState(null)
  const [aviso, setAviso] = useState('')

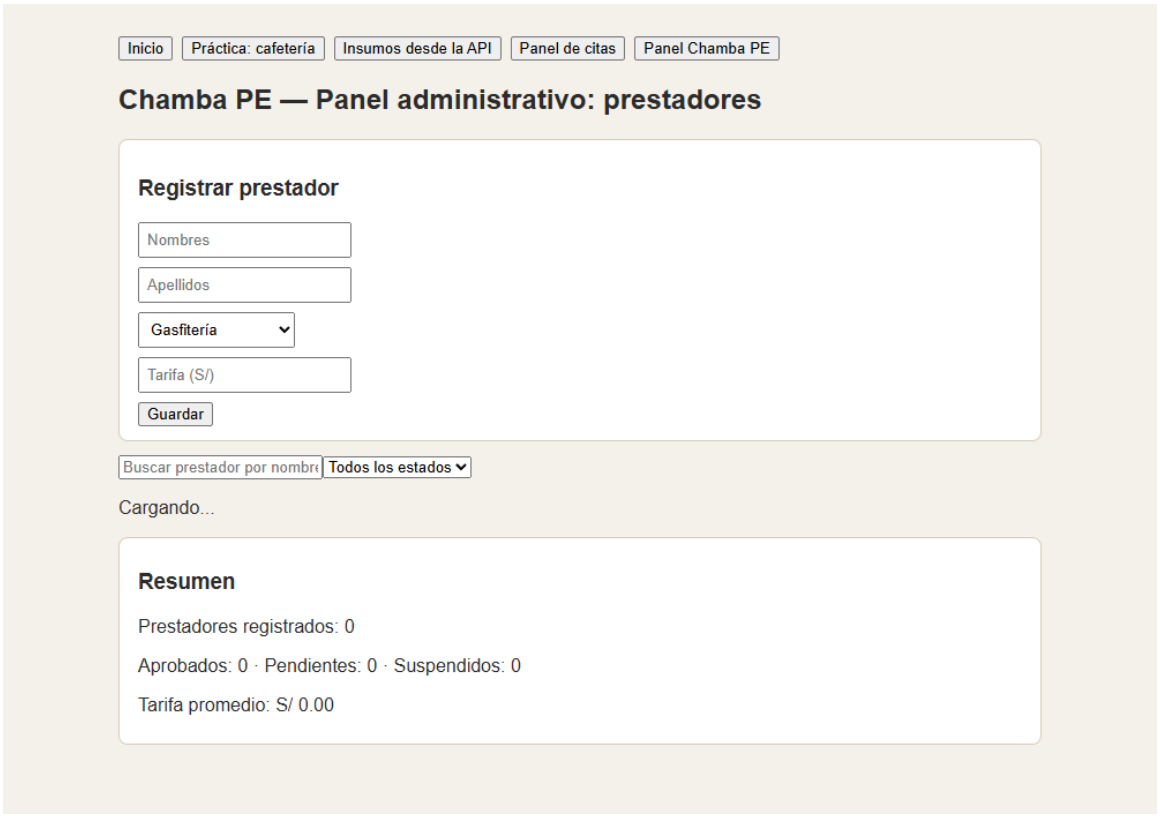
  // Se vuelve a pedir la lista cuando cambian la búsqueda o el filtro
  useEffect(() => {
    const controlador = new AbortController()
    async function cargar() {
      try {
        const respuesta = await api.get('/chamba/prestadores/', {
          params: { search: busqueda, estado: estado },
          signal: controlador.signal,
        })
        setPrestadores(respuesta.data.map(adaptar))
        setError(false)
      } catch (err) {
        if (err.name === 'CanceledError') return
        console.log('Error al cargar los prestadores:', err.message)
        setError(true)
      }
      setCargando(false)
    }
    cargar()
    // Limpieza: cancela la petición pendiente si cambian los parámetros
    return () => controlador.abort()
  }, [busqueda, estado])

  // El título de la pestaña sigue la cantidad de prestadores
  useEffect(() => {
    document.title = `Panel Chamba PE (${prestadores.length})`
  }, [prestadores.length])

  // Devuelve el texto del error de la API, o vacío si todo salió bien
  async function guardarPrestador(datos) {
    try {
      if (editando) {
```




```
Pestaña Network: carga del listado del panel
Método  Est. URL
GET     200  http://127.0.0.1:8000/api/chamba/prestadores/?search=&estado=
```







## Ejercicio 11 — Alta y edición con POST y PUT o PATCH

Haga que el formulario de alta de la Semana 11 envíe un POST a la API y agregue respuesta.data al estado con un arreglo nuevo. Agregue la edición: al presionar Editar en una fila, el padre guarda el registro seleccionado, el formulario se llena con sus datos y al enviarlo se ejecuta api.patch(...) o api.put(...); el estado se actualiza con map(). Muestre al usuario los mensajes de error 400 que envía su serializer. Documente el código, las pantallas y los códigos 201 y 200.

**Respuesta:**

El formulario de alta envía un POST a la API y el prestador nuevo se agrega al estado con un arreglo nuevo, sin recargar la página. Para la edición, al presionar Editar en una fila el padre guarda el prestador seleccionado, el formulario se llena con sus datos y al enviarlo se ejecuta un PATCH; el estado se actualiza con map. Los botones Aprobar y Suspender también envían un PATCH con el nuevo estado. Los errores 400 del serializer se muestran junto al formulario: con una tarifa de cero aparece que la tarifa debe ser mayor que cero. Códigos obtenidos: 201 al crear a Pedro Soto, 400 con la tarifa en cero, y 200 al editar a Pedro (tarifa 48 y rubro Electricidad) y al aprobar a María Torres.

```
# components/PrestadorForm.jsx
import { useState } from 'react'

const formularioVacio = { nombres: '', apellidos: '', rubro: 'Gasfitería', tarifa: '' }

// Formulario controlado para alta y edición. Si recibe un prestador, parte con sus datos
function PrestadorForm({ prestador, onGuardar, onCancelar }) {
  const [form, setForm] = useState(
    prestador
      ? { nombres: prestador.nombres, apellidos: prestador.apellidos, rubro: prestador.rubro, tarifa:
String(prestador.tarifa) }
      : formularioVacio
  )
  const [error, setError] = useState('')

  function manejarCambio(e) {
    setForm({ ...form, [e.target.name]: e.target.value })
  }

  async function manejarEnvio(e) {
    e.preventDefault()
    // Validación previa en el navegador; la API vuelve a validar y responde 400 si algo falla
    if (form.nombres.trim() === '' || form.apellidos.trim() === '') {
      setError('Los nombres y los apellidos son obligatorios.')
      return
    }
    const mensaje = await onGuardar({
      nombres: form.nombres.trim(),
      apellidos: form.apellidos.trim(),
      rubro: form.rubro,
      tarifa: form.tarifa,
    })
    if (mensaje === '') {
      if (!prestador) setForm(formularioVacio)
      setError('')
    } else {
      setError(mensaje)
    }
  }

  return (
    <form onSubmit={manejarEnvio} className="formulario">
      <h3>{prestador ? 'Editar prestador' : 'Registrar prestador'}</h3>
      <input name="nombres" placeholder="Nombres" value={form.nombres} onChange={manejarCambio} />
      <input name="apellidos" placeholder="Apellidos" value={form.apellidos} onChange={manejarCambio} />
      <select name="rubro" value={form.rubro} onChange={manejarCambio}>
        <option>Gasfitería</option>
        <option>Electricidad</option>
        <option>Costura</option>
        <option>Carpintería</option>
        <option>Primeros auxilios</option>
      </select>
      <input name="tarifa" type="number" placeholder="Tarifa (S/)" value={form.tarifa} onChange={manejarCambio} />
      <button type="submit">{prestador ? 'Actualizar' : 'Guardar'}</button>
      {prestador && (
        <button type="button" onClick={onCancelar}>
          Cancelar
        </button>
      )}
      {error && <p className="error">{error}</p>}
    </form>
  )
}

export default PrestadorForm
```

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Registrar prestador

Nombres

Apellidos

Gasfitería

Tarifa (S/)

Guardar

Prestador registrado: Pedro

Buscar prestador por nombr | Todos los estados

| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Pedro Soto Lima | Gasfitería | S/ 45.00 | Pendiente | Editar Aprobar Suspender Eliminar |

### Resumen

Prestadores registrados: 7

Aprobados: 3 · Pendientes: 3 · Suspendidos: 1

Tarifa promedio: S/ 49.29

```
Pestaña Network: POST de un prestador
Método  Est. URL
OPTIONS 200  http://127.0.0.1:8000/api/chamba/prestadores/
POST    201  http://127.0.0.1:8000/api/chamba/prestadores/
```

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Registrar prestador

Julia

Campos Ruiz

Gasfitería

0

Guardar

**tarifa: La tarifa debe ser mayor que cero.**

Prestador registrado: Pedro

Buscar prestador por nombre | Todos los estados

| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Pedro Soto Lima | Gasfitería | S/ 45.00 | Pendiente | Editar Aprobar Suspender Eliminar |

### Resumen

Prestadores registrados: 7

Aprobados: 3 · Pendientes: 3 · Suspendidos: 1

Tarifa promedio: S/ 49.29

```
Pestaña Network: POST con tarifa cero (validate_tarifa)
Método  Est. URL
POST    400  http://127.0.0.1:8000/api/chamba/prestadores/
```

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Editar prestador

Pedro

Soto Lima

Gasfitería

45

Actualizar | Cancelar

Buscar prestador por nombre | Todos los estados

| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Pedro Soto Lima | Gasfitería | S/ 45.00 | Pendiente | Editar Aprobar Suspender Eliminar |

### Resumen

Prestadores registrados: 7

Aprobados: 3 · Pendientes: 3 · Suspendidos: 1

Tarifa promedio: S/ 49.29

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Registrar prestador

Nombres

Apellidos

Gasfitería

Tarifa (S/)

Guardar

Prestador actualizado: Pedro

Buscar prestador por nombre | Todos los estados

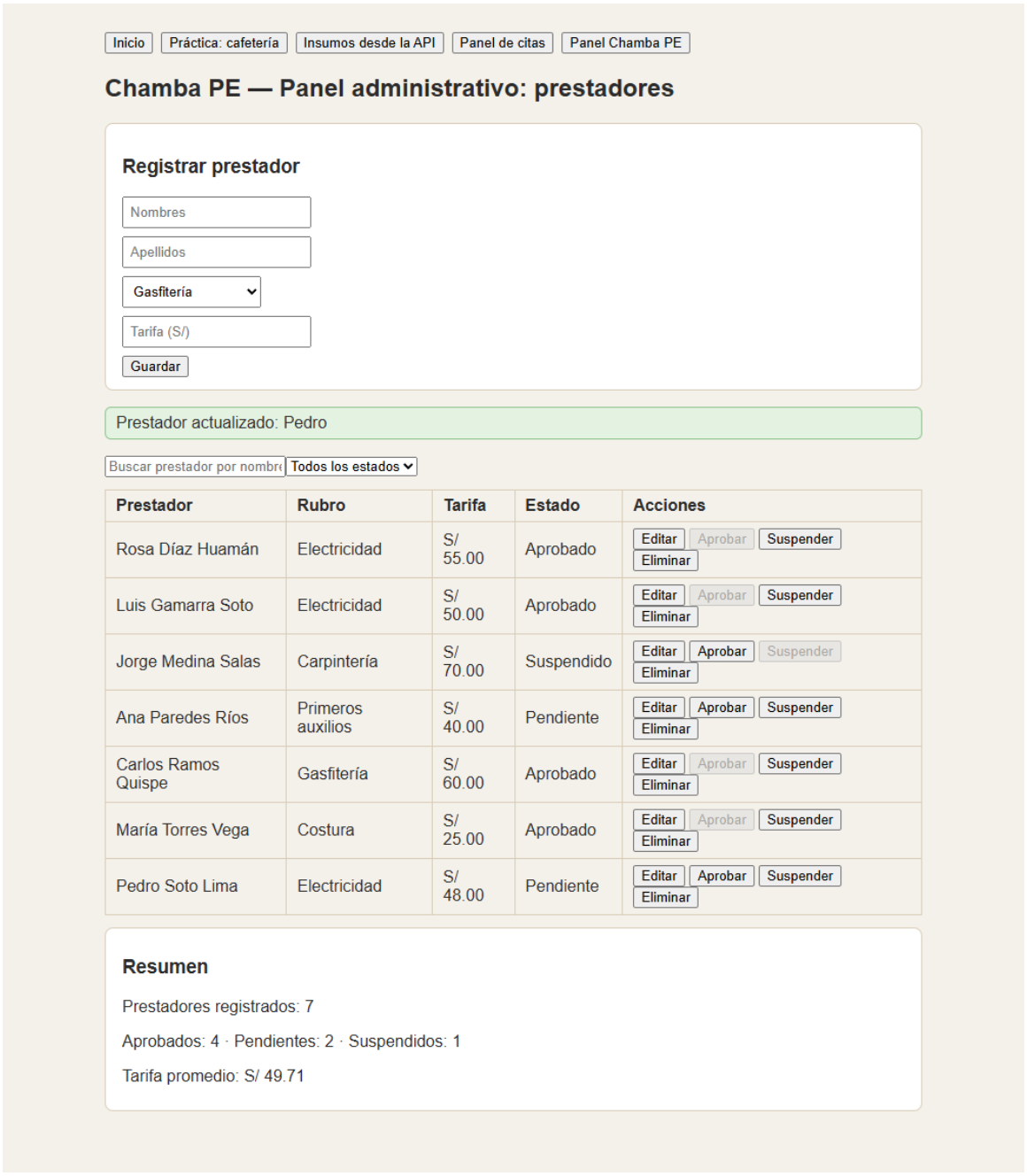
| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Pedro Soto Lima | Electricidad | S/ 48.00 | Pendiente | Editar Aprobar Suspender Eliminar |

### Resumen

Prestadores registrados: 7

Aprobados: 3 · Pendientes: 3 · Suspendidos: 1

Tarifa promedio: S/ 49.71




```
Pestaña Network: PATCH al editar y al aprobar
Método  Est. URL
OPTIONS 200  http://127.0.0.1:8000/api/chamba/prestadores/7/
PATCH   200  http://127.0.0.1:8000/api/chamba/prestadores/7/
OPTIONS 200  http://127.0.0.1:8000/api/chamba/prestadores/2/
PATCH   200  http://127.0.0.1:8000/api/chamba/prestadores/2/
```

## Ejercicio 12 — Eliminar con confirmación

Agregue a cada fila el botón Eliminar con una confirmación previa (por ejemplo, un segundo clic en Confirmar o window.confirm). Al confirmar, ejecute api.delete(...) y quite el registro del estado con filter() cuando reciba el código 204. Maneje el caso en que el servidor responda 404 (el registro ya no existe) mostrando un mensaje. Documente el código y las capturas de Network con los códigos obtenidos.

**Respuesta:**

Cada fila tiene el botón Eliminar con confirmación previa: el primer clic lo cambia por Confirmar y solo el segundo clic ejecuta el DELETE. Al recibir el código 204 el prestador se quita del estado con filter. Si el servidor responde 404 porque el registro ya no existe, el panel muestra un aviso y lo quita de la lista. Para probar el 404 creé un prestador temporal, lo borré directamente en la API y luego pulsé Eliminar y Confirmar en la pantalla, que todavía lo mostraba. En Network se ve antes la petición OPTIONS, que es la consulta previa de CORS.

```
# components/PanelPrestadores.jsx
import { useEffect, useState } from 'react'
import api, { textoError } from '../api.js'
import PrestadorListado from './PrestadorListado.jsx'
import PrestadorForm from './PrestadorForm.jsx'
import ResumenPrestadores from './ResumenPrestadores.jsx'
import BuscadorPrestadores from './BuscadorPrestadores.jsx'
import FiltroEstadoPrestador from './FiltroEstadoPrestador.jsx'
import Aviso from './Aviso.jsx'

// Función adaptadora: la API entrega la tarifa como texto ("60.00") y el panel la usa como número
function adaptar(prestador) {
  return { ...prestador, tarifa: Number(prestador.tarifa) }
}

// Componente padre del panel de Chamba PE: guarda los prestadores y los estados de la petición
function PanelPrestadores() {
  const [prestadores, setPrestadores] = useState([])
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState(false)
  const [busqueda, setBusqueda] = useState('')
  const [estado, setEstado] = useState('')
  const [editando, setEditando] = useState(null)
  const [porEliminar, setPorEliminar] = useState(null)
  const [aviso, setAviso] = useState('')

  // Se vuelve a pedir la lista cuando cambian la búsqueda o el filtro
  useEffect(() => {
    const controlador = new AbortController()
    async function cargar() {
      try {
        const respuesta = await api.get('/chamba/prestadores/', {
          params: { search: busqueda, estado: estado },
          signal: controlador.signal,
        })
        setPrestadores(respuesta.data.map(adaptar))
        setError(false)
      } catch (err) {
        if (err.name === 'CanceledError') return
        console.log('Error al cargar los prestadores:', err.message)
        setError(true)
      }
      setCargando(false)
    }
    cargar()
    // Limpieza: cancela la petición pendiente si cambian los parámetros
    return () => controlador.abort()
  }, [busqueda, estado])

  // El título de la pestaña sigue la cantidad de prestadores
  useEffect(() => {
    document.title = `Panel Chamba PE (${prestadores.length})`
  }, [prestadores.length])

  // Devuelve el texto del error de la API, o vacío si todo salió bien
  async function guardarPrestador(datos) {
    try {
      if (editando) {
        const respuesta = await api.patch(`/chamba/prestadores/${editando.id}/`, datos)
        setPrestadores(prestadores.map((p) => (p.id === editando.id ? adaptar(respuesta.data) : p)))
        setAviso(`Prestador actualizado: ${datos.nombres}`)
        setEditando(null)
      } else {
        const respuesta = await api.post('/chamba/prestadores/', datos)
        setPrestadores([...prestadores, adaptar(respuesta.data)])
        setAviso(`Prestador registrado: ${datos.nombres}`)
      }
      return ''
    } catch (err) {
      return textoError(err)
    }
  }

  async function cambiarEstado(prestador, nuevoEstado) {
    try {
      const respuesta = await api.patch(`/chamba/prestadores/${prestador.id}/`, { estado: nuevoEstado })
      setPrestadores(prestadores.map((p) => (p.id === prestador.id ? adaptar(respuesta.data) : p)))
    } catch (err) {
      setAviso(textoError(err))
    }
```

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Registrar prestador

Nombres

Apellidos

Gasfitería

Tarifa (S/)

Guardar

Prestador actualizado: Pedro

Buscar prestador por nombre | Todos los estados

| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Pedro Soto Lima | Electricidad | S/ 48.00 | Pendiente | Editar Aprobar Suspender Eliminar |

### Resumen

Prestadores registrados: 7

Aprobados: 4 · Pendientes: 2 · Suspendidos: 1

Tarifa promedio: S/ 49.71

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Registrar prestador

Nombres

Apellidos

Gasfitería

Tarifa (S/)

Guardar

Prestador actualizado: Pedro

Buscar prestador por nombre | Todos los estados

| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Pedro Soto Lima | Electricidad | S/ 48.00 | Pendiente | Editar Aprobar Suspender Confirmar |

### Resumen

Prestadores registrados: 7

Aprobados: 4 · Pendientes: 2 · Suspendidos: 1

Tarifa promedio: S/ 49.71

Inicio | Práctica: cafetería | Insumos desde la API | Panel de citas | Panel Chamba PE

## Chamba PE — Panel administrativo: prestadores

### Registrar prestador

Nombres

Apellidos

Gasfitería

Tarifa (S/)

Guardar

Prestador eliminado

Buscar prestador por nombre | Todos los estados

| Prestador | Rubro | Tarifa | Estado | Acciones |
|---|---|---|---|---|
| Rosa Díaz Huamán | Electricidad | S/ 55.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Luis Gamarra Soto | Electricidad | S/ 50.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| Jorge Medina Salas | Carpintería | S/ 70.00 | Suspendido | Editar Aprobar Suspender Eliminar |
| Ana Paredes Ríos | Primeros auxilios | S/ 40.00 | Pendiente | Editar Aprobar Suspender Eliminar |
| Carlos Ramos Quispe | Gasfitería | S/ 60.00 | Aprobado | Editar Aprobar Suspender Eliminar |
| María Torres Vega | Costura | S/ 25.00 | Aprobado | Editar Aprobar Suspender Eliminar |

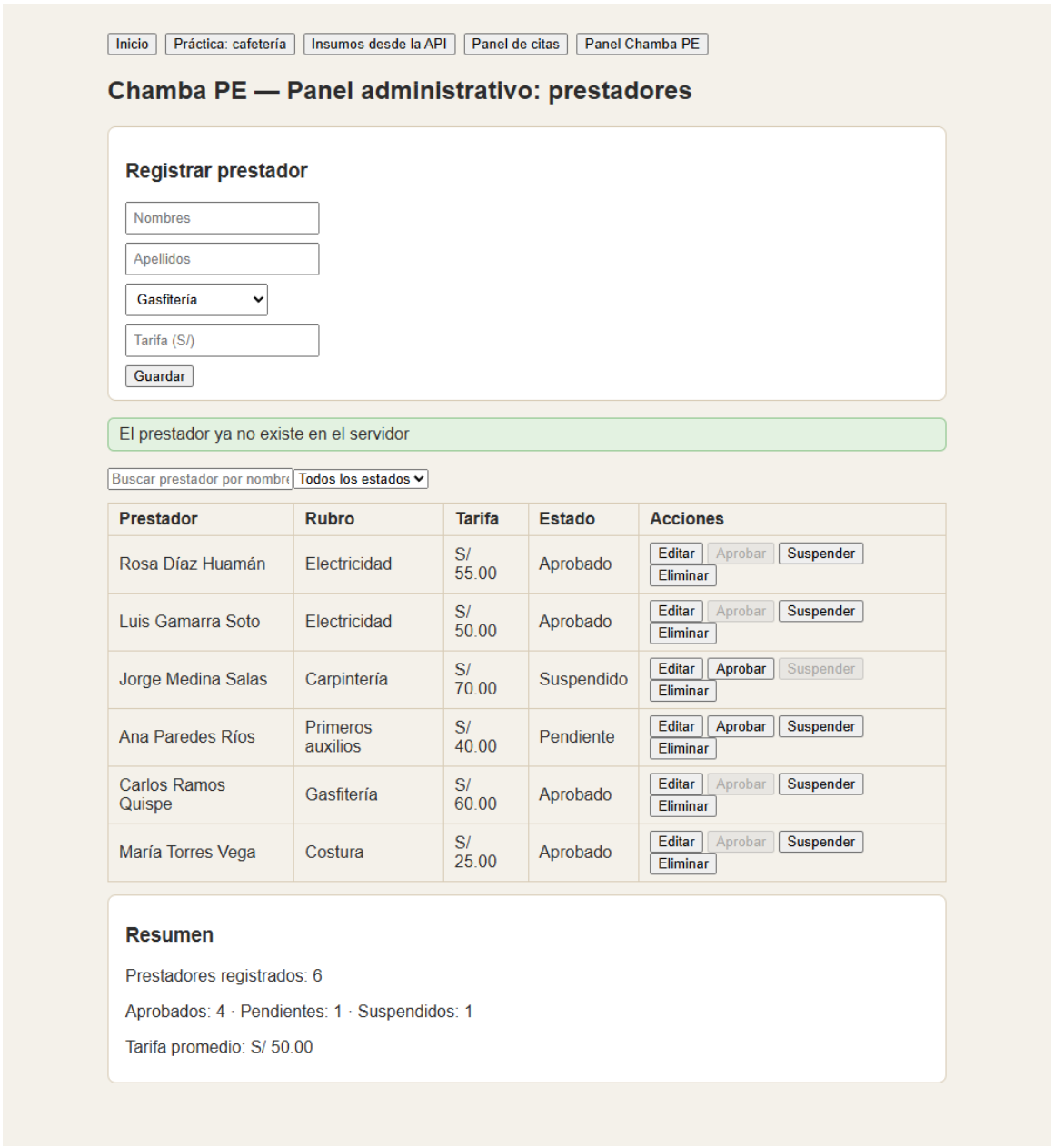
### Resumen

Prestadores registrados: 6

Aprobados: 4 · Pendientes: 1 · Suspendidos: 1

Tarifa promedio: S/ 50.00

```
Pestaña Network: DELETE de un prestador (204)
Método  Est. URL
DELETE  204  http://127.0.0.1:8000/api/chamba/prestadores/7/
```




```
Pestaña Network: DELETE de un prestador que ya no existe (404)
Método  Est. URL
OPTIONS 200  http://127.0.0.1:8000/api/chamba/prestadores/8/
DELETE  404  http://127.0.0.1:8000/api/chamba/prestadores/8/
```

## Ejercicio 13 — Búsqueda, filtros y resumen con datos de la API

Agregue al panel un buscador y un filtro por un campo (por ejemplo, el estado) que envíen sus valores como parámetros a la API y recarguen el listado cuando cambien. Haga que el resumen de la Semana 11 (total, conteo o promedio) se calcule con los datos recibidos de la API. Documente al menos dos URLs generadas con sus resultados, una de ellas combinando parámetros con &. Opcional: cancele la petición pendiente con AbortController en la limpieza del efecto.

**Respuesta:**

El panel tiene un buscador por nombre y un filtro por estado que se envían como parámetros a la API, y el listado se vuelve a pedir cuando cambian. El resumen calcula el total, el conteo por estado y la tarifa promedio con los datos recibidos de la API. Como parte opcional, la limpieza del efecto cancela la petición

pendiente con AbortController. Direcciones generadas: http://127.0.0.1:8000/api/chamba/prestadores/?search=ro&estado= devuelve a Rosa Díaz y a Pedro Soto; http://127.0.0.1:8000/api/chamba/prestadores/?search=&estado=Aprobado devuelve solo los aprobados; y http://127.0.0.1:8000/api/chamba/prestadores/?search=a&estado=Aprobado combina los dos parámetros con el símbolo &. Con zzz la API no devuelve nada y aparece el mensaje.

```
# components/ResumenPrestadores.jsx
// Resumen hermano del listado: todo se calcula con los prestadores recibidos, sin estado propio
function ResumenPrestadores({ prestadores }) {
  const aprobados = prestadores.filter((p) => p.estado === 'Aprobado').length
  const pendientes = prestadores.filter((p) => p.estado === 'Pendiente').length
  const suspendidos = prestadores.filter((p) => p.estado === 'Suspendido').length
  const promedio =
    prestadores.length === 0
      ? 0
      : prestadores.reduce((suma, p) => suma + Number(p.tarifa), 0) / prestadores.length

  return (
    <div className="panel-resumen">
      <h3>Resumen</h3>
      <p>Prestadores registrados: {prestadores.length}</p>
      <p>Aprobados: {aprobados} · Pendientes: {pendientes} · Suspendidos: {suspendidos}</p>
      <p>Tarifa promedio: S/ {promedio.toFixed(2)}</p>
    </div>
  )
}

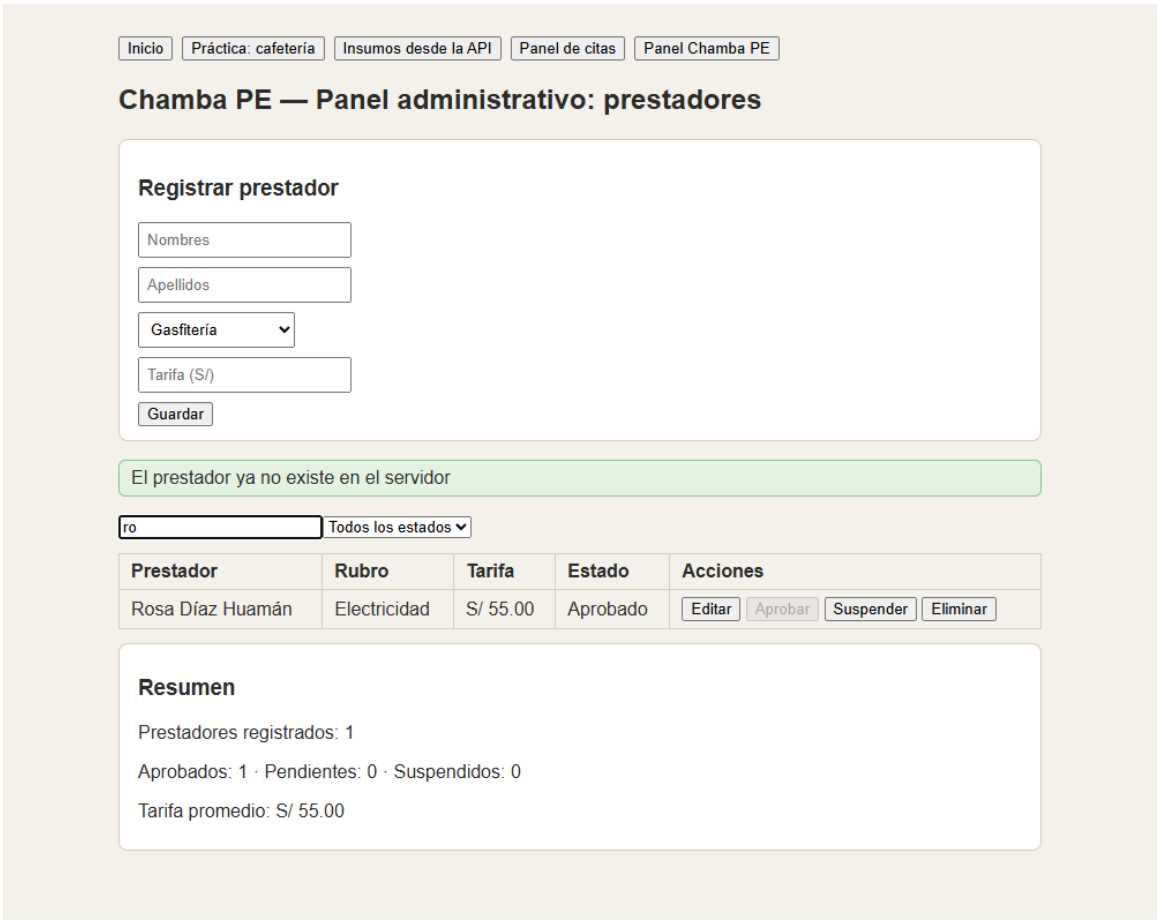
export default ResumenPrestadores

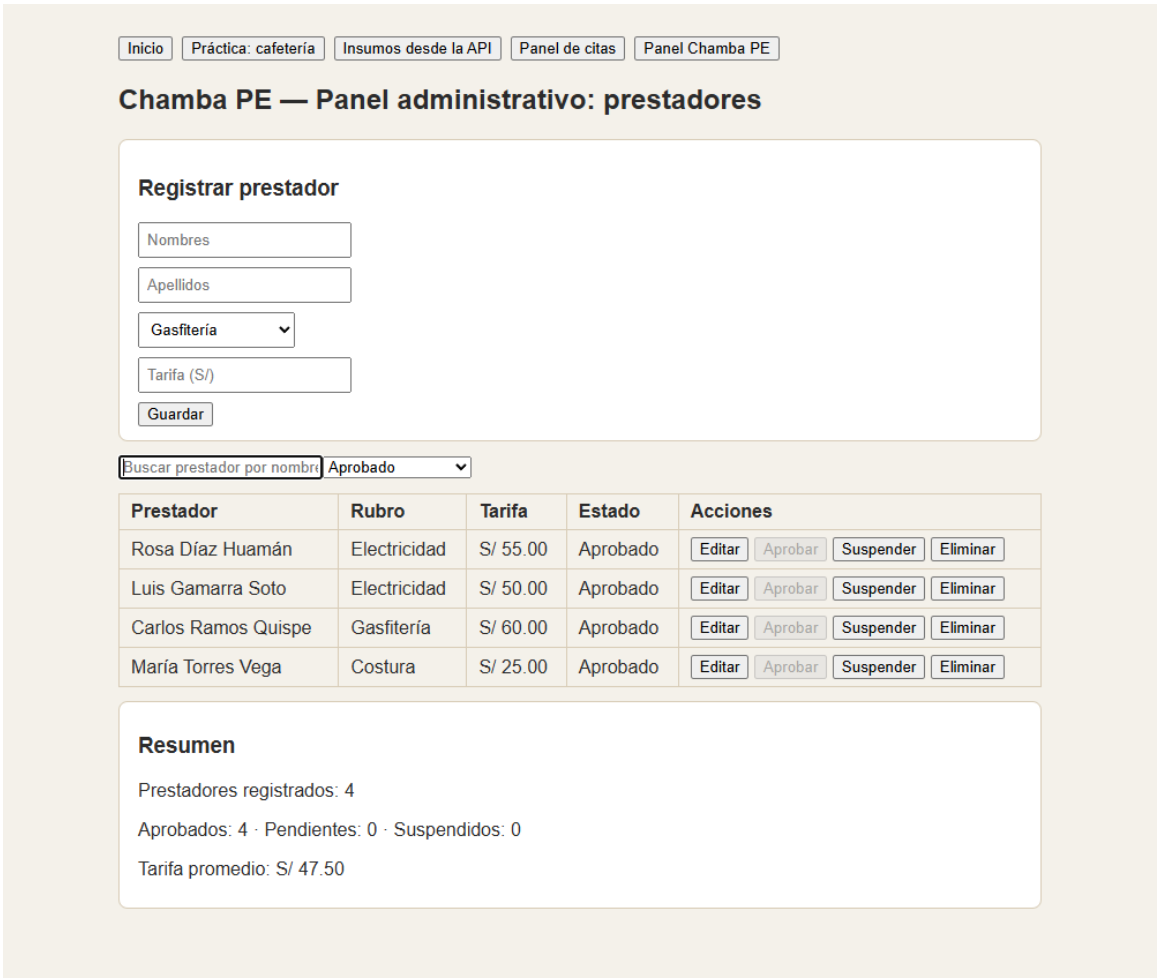
# components/BuscadorPrestadores.jsx
// Filtro de texto controlado: el valor y el cambio los maneja el padre
function BuscadorPrestadores({ valor, onCambiar }) {
  return (
    <input
      type="text"
      placeholder="Buscar prestador por nombre"
      value={valor}
      onChange={(e) => onCambiar(e.target.value)}
    />
  )
}

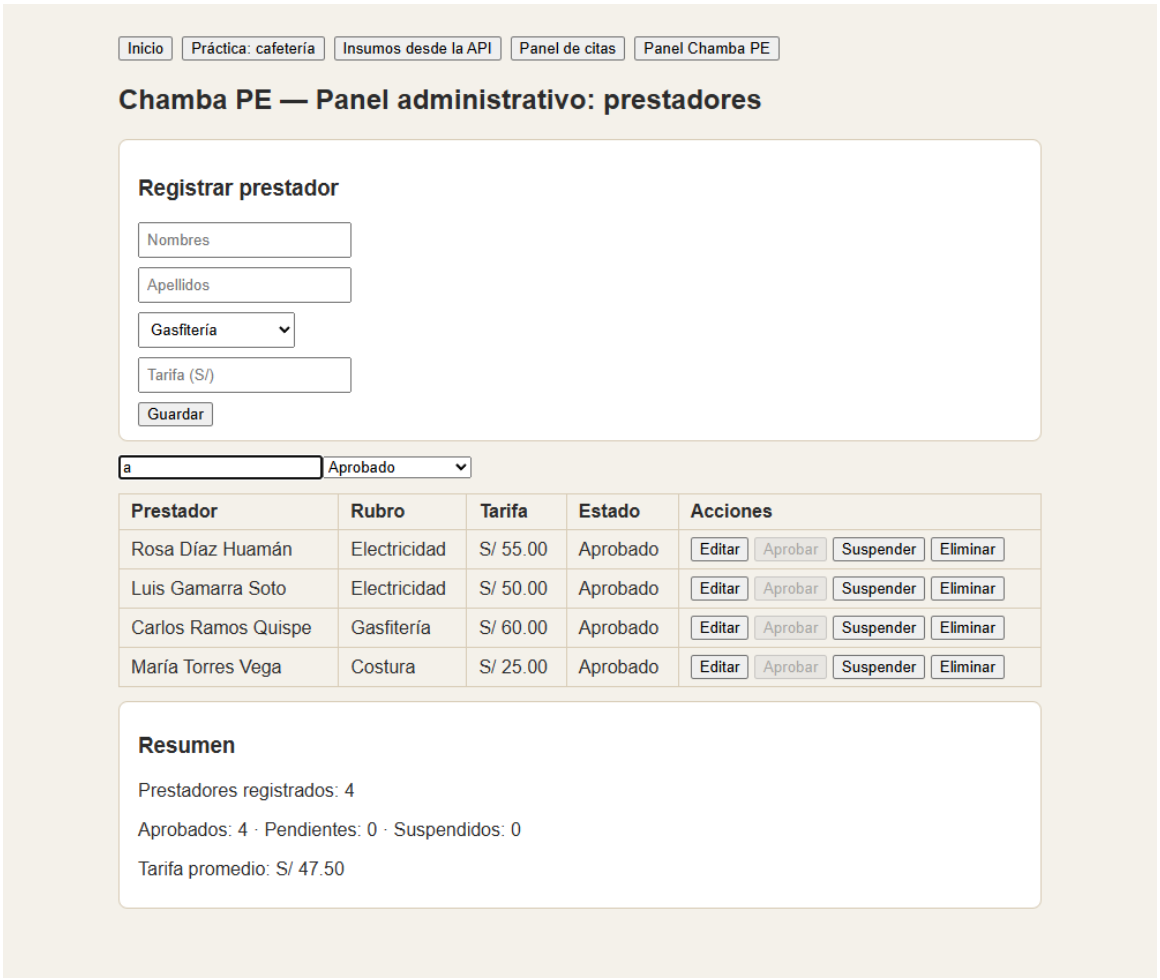
export default BuscadorPrestadores

# components/FiltroEstadoPrestador.jsx
// Filtro por estado controlado por el padre
function FiltroEstadoPrestador({ valor, onCambiar }) {
  return (
    <select value={valor} onChange={(e) => onCambiar(e.target.value)}>
      <option value="">Todos los estados</option>
      <option>Pendiente</option>
      <option>Aprobado</option>
      <option>Suspendido</option>
    </select>
  )
}

export default FiltroEstadoPrestador
```


Inicio
Práctica: cafetería
Insumos desde la API
Panel de citas
Panel Chamba PE
Chamba PE — Panel administrativo: prestadores
Registrar prestador
Nombres
Apellidos
Gasfitería
Tarifa (S/)
Guardar
El prestador ya no existe en el servidor
ro
Todos los estados
Prestador
Rubro
Tarifa
Estado
Acciones
Rosa Díaz Huamán
Electricidad
S/ 55.00
Aprobado
Editar
Aprobar
Suspender
Eliminar
Resumen
Prestadores registrados: 1
Aprobados: 1 · Pendientes: 0 · Suspendidos: 0
Tarifa promedio: S/ 55.00


Inicio
Práctica: cafetería
Insumos desde la API
Panel de citas
Panel Chamba PE
Chamba PE — Panel administrativo: prestadores
Registrar prestador
Nombres
Apellidos
Gasfitería
Tarifa (S/)
Guardar
Buscar prestador por nombre
Aprobado
Prestador
Rubro
Tarifa
Estado
Acciones
Rosa Díaz Huamán
Electricidad
S/ 55.00
Aprobado
Editar
Aprobar
Suspender
Eliminar
Luis Gamarra Soto
Electricidad
S/ 50.00
Aprobado
Editar
Aprobar
Suspender
Eliminar
Carlos Ramos Quispe
Gasfitería
S/ 60.00
Aprobado
Editar
Aprobar
Suspender
Eliminar
María Torres Vega
Costura
S/ 25.00
Aprobado
Editar
Aprobar
Suspender
Eliminar
Resumen
Prestadores registrados: 4
Aprobados: 4 · Pendientes: 0 · Suspendidos: 0
Tarifa promedio: S/ 47.50




```
Pestaña Network: búsqueda, filtro por estado y combinación con &
Método  Est. URL
GET     200  http://127.0.0.1:8000/api/chamba/prestadores/?search=r&estado=
GET     200  http://127.0.0.1:8000/api/chamba/prestadores/?search=ro&estado=
GET     200  http://127.0.0.1:8000/api/chamba/prestadores/?search=&estado=
GET     200  http://127.0.0.1:8000/api/chamba/prestadores/?search=&estado=Aprobado
GET     200  http://127.0.0.1:8000/api/chamba/prestadores/?search=a&estado=Aprobado
```

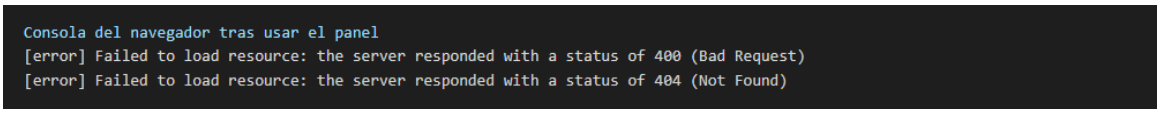



## Ejercicio 14 — Verificar y publicar en GitHub

Verifique que la consola del navegador no muestra advertencias, que la terminal de Django no muestra errores 500, y ejecute npm run lint y npm run build. Compruebe que node_modules y dist no están en el repositorio. Redacte un párrafo que explique qué resuelve Ajax frente a los Templates en su panel y qué le falta todavía a la aplicación (por ejemplo, la autenticación y los permisos). Actualice el README.md con los pasos para ejecutar Django y React a la vez, la configuración de CORS y los endpoints que usa el panel. Haga commit y push, e incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

**Respuesta:**

La consola del navegador no muestra advertencias, la terminal de Django no muestra errores 500, npm run lint no reporta errores y npm run build termina correctamente. Las carpetas node_modules y dist están ignoradas y no se suben. Ajax resuelve que el panel se actualice sin recargar la página: en lugar de pedir una página nueva a un Template en cada acción, React pide solo los datos a la API y cambia lo necesario en pantalla, mientras Django se encarga de validar y de guardar en la base de datos. A la aplicación todavía le falta la autenticación y los permisos: hoy cualquiera que conozca la dirección puede crear, editar o eliminar prestadores, y en Chamba PE ese panel solo debería usarlo el administrador. El README explica cómo ejecutar Django y React a la vez, la configuración de CORS y los endpoints que usa el panel. Repositorio: https://github.com/RXDG2908/DesAplEmp-S02



```
PS ...> python manage.py runserver 8000
[14/Nov/2026 05:56:37] "GET /api/chamba/prestadores/?search=&estado= HTTP/1.1" 200 687
[14/Nov/2026 05:56:41] "OPTIONS /api/chamba/prestadores/ HTTP/1.1" 200 0
[14/Nov/2026 05:56:41] "POST /api/chamba/prestadores/ HTTP/1.1" 201 110
Bad Request: /api/chamba/prestadores/
[14/Nov/2026 05:56:43] "POST /api/chamba/prestadores/ HTTP/1.1" 400 49
[14/Nov/2026 05:56:47] "OPTIONS /api/chamba/prestadores/7/ HTTP/1.1" 200 0
[14/Nov/2026 05:56:47] "PATCH /api/chamba/prestadores/7/ HTTP/1.1" 200 111
[14/Nov/2026 05:56:48] "OPTIONS /api/chamba/prestadores/2/ HTTP/1.1" 200 0
[14/Nov/2026 05:56:48] "PATCH /api/chamba/prestadores/2/ HTTP/1.1" 200 108
[14/Nov/2026 05:56:49] "DELETE /api/chamba/prestadores/7/ HTTP/1.1" 204 0
[14/Nov/2026 05:56:52] "POST /api/chamba/prestadores/ HTTP/1.1" 201 114
[14/Nov/2026 05:56:52] "GET /api/chamba/prestadores/?search=Temporal HTTP/1.1" 200 116
[14/Nov/2026 05:56:52] "DELETE /api/chamba/prestadores/8/ HTTP/1.1" 204 0
[14/Nov/2026 05:56:53] "OPTIONS /api/chamba/prestadores/8/ HTTP/1.1" 200 0
Not Found: /api/chamba/prestadores/8/
[14/Nov/2026 05:56:53] "DELETE /api/chamba/prestadores/8/ HTTP/1.1" 404 50
[14/Nov/2026 05:56:54] "GET /api/chamba/prestadores/?search=r&estado= HTTP/1.1" 200 686
[14/Nov/2026 05:56:54] "GET /api/chamba/prestadores/?search=ro&estado= HTTP/1.1" 200 115
[14/Nov/2026 05:56:55] "GET /api/chamba/prestadores/?search=&estado= HTTP/1.1" 200 686
[14/Nov/2026 05:56:56] "GET /api/chamba/prestadores/?search=&estado=Aprobado HTTP/1.1" 200 451
[14/Nov/2026 05:56:57] "GET /api/chamba/prestadores/?search=a&estado=Aprobado HTTP/1.1" 200 451
[14/Nov/2026 05:56:58] "GET /api/chamba/prestadores/?search=zzz&estado=Aprobado HTTP/1.1" 200 2
[14/Nov/2026 05:56:59] "GET /api/chamba/prestadores/?search=&estado=Aprobado HTTP/1.1" 200 451
[14/Nov/2026 05:57:00] "GET /api/chamba/prestadores/?search=&estado= HTTP/1.1" 200 686

Respuestas 500 de Django: 0
```

```
PS ...> npm run lint

> frontend@0.0.0 lint
> eslint .
```

```
PS ...> npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.5 building client environment for production...
transforming...
✓ 110 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.47 kB │ gzip:  0.30 kB
dist/assets/index-xrUpEBpY.css    1.27 kB │ gzip:  0.47 kB
dist/assets/index-DXocnDlM.js   302.69 kB │ gzip: 96.45 kB

✓ built in 159ms
```

```
PS ...> git check-ignore -v node_modules dist
frontend/.gitignore:10:node_modules     node_modules
frontend/.gitignore:11:dist     dist
Archivos de node_modules o dist en el repositorio: 0
```

## Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente api.js, los componentes, la pestaña Network con el método y el código de estado, y la salida de npm run lint y npm run build).
- Repositorio en GitHub actualizado, con la carpeta frontend, los cambios de settings.py y requirements.txt, el README.md y el código correspondiente, sin node_modules ni dist.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

**Repositorio: https://github.com/RXDG2908/DesAplEmp-S02**

## Conclusiones:

| |
|---|
| Django entrega y valida los datos con serializers y ViewSets, y React los pide con una petición Ajax y los dibuja. |
| CORS es una protección del navegador: hay que autorizar el origen de React en Django para que pueda leer las respuestas. |
| Axios con una instancia única deja la dirección del servidor en un solo archivo. |
| La petición GET va dentro de un efecto, y los tres estados, cargando, error y lista vacía, evitan dejar la pantalla en blanco. |
| POST, PATCH y DELETE actualizan el estado con arreglos nuevos solo cuando la API responde bien, y los errores 400 y 404 se muestran al usuario. |
| Los parámetros de búsqueda y de filtro se envían a la API y el listado se recarga cuando cambian. |
| La API sigue abierta a cualquier persona: falta la autenticación y los permisos. |